

Language Models and RNNs

SMM694: Natural Language Processing

Session 4 | Valerio Modugno

Lecture Overview & Roadmap

01 Introduction to Language Modeling

02 RNN-based Language Models

03 Vanishing Gradients & RNN Issues

04 RNN Extensions

05 Machine Translation

06 Intro to Attention

Defining Language Modeling

At its core, **Language Modeling** is the task of predicting the next word in a sequence. Given a sequence of words, the model computes a probability distribution over the entire vocabulary.

$$P(\mathbf{x}^{(t+1)} \mid \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)})$$

Example Scenario

"the students opened their..."

The model predicts: **books** (0.4), **laptops** (0.3), **exams** (0.1)...

Scoring Natural Language

Sequence Probability

Language Models don't just predict the *next* word; they can assign a probability to an entire piece of text. This "score" determines how natural or likely a sentence is in a given language.

By applying the **Chain Rule of Probability**, we can decompose the joint probability of a whole sentence into a product of conditional probabilities.

$$\begin{aligned} P(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}) &= P(\mathbf{x}^{(1)}) \times P(\mathbf{x}^{(2)} | \mathbf{x}^{(1)}) \times \dots \times P(\mathbf{x}^{(T)} | \mathbf{x}^{(T-1)}, \dots, \mathbf{x}^{(1)}) \\ &= \prod_{t=1}^T P(\mathbf{x}^{(t)} | \mathbf{x}^{(t-1)}, \dots, \mathbf{x}^{(1)}) \end{aligned}$$

The product of consecutive probabilities scores the sequence.

LMs in Everyday Technology



Predictive Typing

Smartphone keyboards use language models to suggest the next word based on current typing patterns and personal history.



Autocomplete

Google Search's autocomplete feature uses massive LMs to predict the search query intent as you type the first few characters.



Assistive Writing

Email platforms suggest quick replies or complete sentences (e.g., Smart Compose) using real-time LM inference.

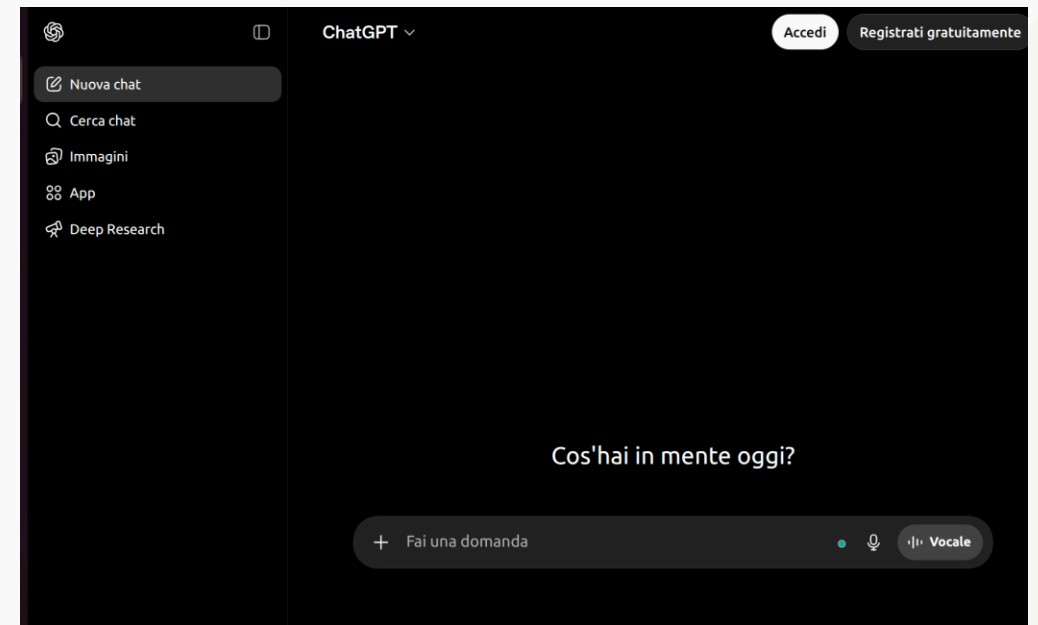


State-of-the-Art: ChatGPT

A Conversational Breakthrough

ChatGPT is a prime example of a modern, large-scale language model. While it feels "intelligent," it is fundamentally optimized for a single task: **dialogue**.

- Trained on massive datasets to predict tokens in a way that is helpful and coherent.
- Fine-tuned using human feedback (RLHF) to follow instructions.
- Subject to limitations like factual hallucinations and logic errors.



What a Strong LM Knows

The Learning Process

Predicting the next word forces the model to learn underlying patterns that it wasn't explicitly taught.

To guess correctly, the model must "understand" the world to some degree.

✓ Syntax & Grammar

🧠 World Trivia & Facts

✂️ Coreference & Context

📊 Basic Arithmetic

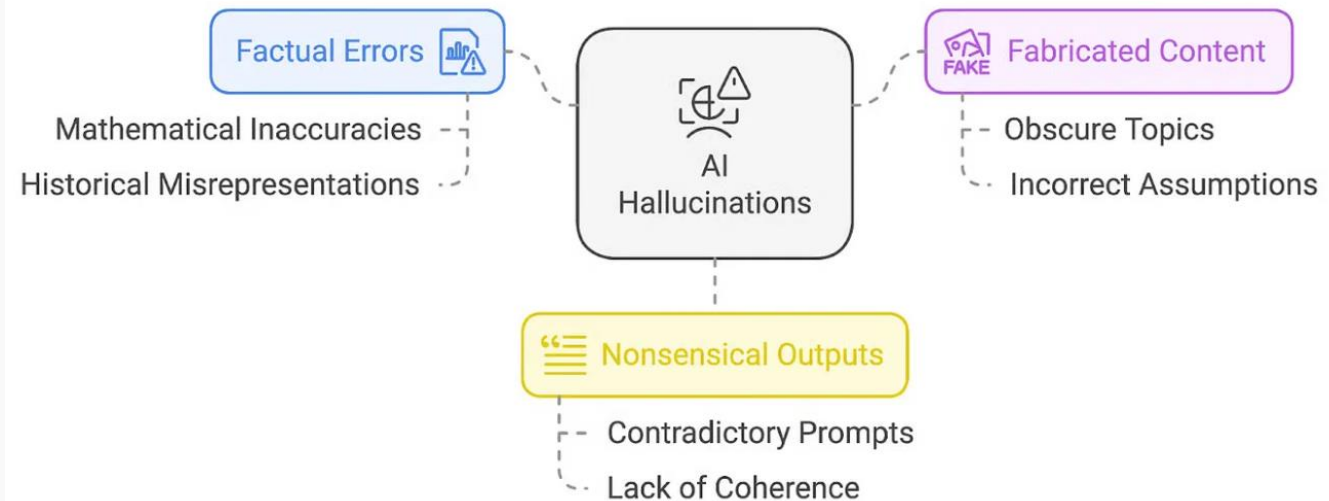
GPT-2: The 2019 Milestone

In 2019, OpenAI released GPT-2, which demonstrated that LMs could generate long, stylistically perfect paragraphs of text.

Observation: While the generated text was highly coherent and sounded human, it was often factually questionable or logically circular.

It marked a shift from "predicting words" to "generating creative content" at scale.

Understanding AI Hallucinations: Types and Examples



What Makes Good In-Context Examples for GPT-3?

Jiachang Liu¹; Dinghan Shen², Yizhe Zhang³, Bill Dolan⁴, Lawrence Carin¹, Weizhu Chen²

¹Duke University ²Microsoft Dynamics 365 AI ³Meta AI ⁴Microsoft Research

¹{jiachang.liu, lcarin}@duke.edu

³yizhe.zhang@hotmail.com

^{2,4}{dishen, billdol, wzchen}@microsoft.com

Abstract

GPT-3 has attracted lots of attention due to its superior performance across a wide range of NLP tasks, especially with its in-context learning abilities. Despite its success, we found that the empirical results of GPT-3 depend heavily on the choice of in-context examples. In this work, we investigate whether there are more effective strategies for judiciously selecting in-context examples (relative to random sampling) that better leverage GPT-3's in-context learning capabilities. Inspired by the recent success of leveraging a retrieval module to augment neural networks, we propose to retrieve examples that are semantically-similar to a test query sample to formulate its corresponding prompt. Intuitively, the examples selected with such a strategy may serve as more informative inputs to unleash GPT-3's power of text generation. We evaluate the proposed approach on several natural language understanding and generation benchmarks, where the retrieval-based prompt selection approach consistently outperforms the random selection baseline. Moreover, it is observed that the sentence encoders fine-tuned on task-related datasets yield even more helpful retrieval results. Notably, significant gains are observed on tasks such as table-to-text generation (44.3% on the ToTTo dataset) and open-domain question answering (45.5% on the NQ dataset).

Trial	1	2	3	4	5
Accuracy	94.6	95.0	95.8	93.9	86.9

Table 1: Results of GPT-3 on the SST-2 sentiment analysis dataset. Five different examples are randomly selected from the training set for each trial. Different contexts induce different accuracies on the test set.

to human. Future NLP systems can be developed to expand emails, extract entities from text, generate code based on natural language instructions with a few demonstration examples.

Despite its powerful and versatile in-context learning ability, GPT-3 has some practical challenges. The original paper utilizes task-relevant examples that are randomly sampled from the training set. However, we observe that the performance of GPT-3 tends to fluctuate with different choices of in-context examples. As shown in Table 1, the variance with distinct in-context examples can be significant. Our work aims to carefully examine this issue to gain a deeper understanding on how to better select in-context examples to improve GPT-3's performance without fine-tuning. Note that our approach requires a training set to select examples. With such a training dataset, it is possible to fine-tune GPT-3 to take full advantage of the model's strength. However, currently GPT-3 has not been released to public for fine-tuning. Even if it is available, fine-tuning GPT-3 requires hun-

GPT-3 & In-Context Learning

By 2020, GPT-3 introduced a radical new capability: **In-context learning**. The model became so massive that it could solve tasks it was never explicitly trained for.

The Blitz Example: By simply seeing a few question-answer pairs in the prompt, the model "learns" to perform Q&A on a completely new text without any gradient updates.

Traditional Foundations

The Era of N-gram Language Models

Before Deep Learning, language models relied on counting word sequences in massive text corpora.

N-grams & Markov Assumption

The Markov Approximation

Counting all possible sentences is impossible. We simplify the problem by assuming the next word depends only on the previous $n-1$ words.

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \underbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}_{n-1 \text{ words}})$$



$$\approx \frac{\text{count}(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}{\text{count}(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})}$$

(statistical approximation)

Unigram: 1-word chunks

Bigram: 2-word chunks

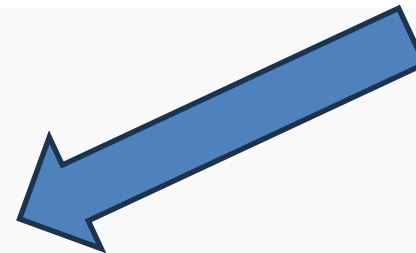
Trigram: 3-word chunks

Four-gram: 4-word chunks

prob of a n-gram $\rightarrow P(\mathbf{x}^{(t+1)}, \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})$

=

prob of a (n-1)-gram $\rightarrow P(\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)})$



Calculating Probability by Counting

Suppose we are learning a 4-gram Language Model.

~~as the proctor started the clock, the~~ students opened their _____
discard condition on this

$$P(\boldsymbol{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

For example, suppose that in the corpus:

- “students opened their” occurred 1000 times
 - “students opened their books” occurred 400 times
 - $\rightarrow P(\text{books} | \text{students opened their}) = 0.4$
 - “students opened their exams” occurred 100 times
 - $\rightarrow P(\text{exams} | \text{students opened their}) = 0.1$
- Should we have discarded the “proctor” context?

The Sparsity Problem

1. Missing N-grams

What if the specific word combination never occurred in our training data? The count is 0, and the probability becomes 0.

Solution: **Smoothing** (adding a small δ to all counts).

2. Missing Context

What if the context itself ("students opened their") never occurred? We cannot divide by zero.

Solution: **Backoff** (reducing n and looking at trigrams instead).

The Storage Problem

As we increase n (to capture more context) or increase training data, the number of unique word combinations explodes exponentially.

Key Takeaway

N-gram models are simple but fundamentally unscalable. They cannot generalize to never-before-seen sentences, leading to the necessity of **Neural Language Models** (RNNs), which we will cover next.



Practical Trigram Example: Reuters Corpus

Let's observe a practical application of a count-based model trained on a real-world dataset: a corpus of Reuters financial news.

By counting occurrences, the model generates a probability distribution for the token following the sequence "**today the**_____".

The Granularity Problem

While simple count-based models work decently for basic, highly-frequent local predictions in narrow domains (like predicting financial nouns), they inherently lack flexibility. Their probabilities are strictly bound to exact matches found in the training text.

Context	Next Word (\$w_t\$)	Probability
today the	company	0.150
today the	bank	0.150
today the	price	0.082
today the	market	0.054
today the	stock	0.041

Text Generation: Step 1

Language models generate text iteratively, in an auto-regressive manner. In this first step, the model evaluates its calculated probability distribution and **samples** a word to become the next token in the sequence.

today the price

The Sampling Mechanism

Rather than always picking the absolute most likely word (which can lead to repetitive, robotic text), we often sample from the distribution. Here, the word "**price**" was randomly drawn based on its 8.2% statistical likelihood following "today the".

Text Generation: Step 2

In the second step of generation, the context window shifts forward. Because this is a **trigram model** ($n=3$), it can only ever look at the immediately preceding $n-1 = 2$ words.

today **the price** [of]

of	0.308	sample
for	0.050	
it	0.046	
to	0.046	
is	0.031	
...		

Context Discarded

The word "today" is now completely ignored by the model. It plays no mathematical role in predicting the next word.

New Prediction

The model consults its database for the counts of words following "the price", leading it to sample the highly probable preposition **"of"**.

Text Generation: Step 3

The third step clearly highlights the iterative, step-by-step nature of auto-regressive language generation. The output produced from the previous step seamlessly becomes the input context for the current step.

today the **price of** [gold]

the	0.072
18	0.043
oil	0.043
its	0.036
gold	0.018
...	

sample

The window shifts once again. Now both "today" and "the" fall completely outside the trigram's memory. The model relies solely on the bigram context "**price of**" to sample the financial noun "**gold**".

The Breaking Point of N-Grams

Locally Grammatical, Globally Incoherent

If we let this trigram model run continuously, it produces a paragraph that feels structurally sound over any 3-word span, but **is complete gibberish** when read as a whole.

- **The Core Flaw:** Because the n-gram model only looks at a tight 3-word window, it forgets the main subject of its own sentence almost instantly.
- **The Sparsity Limit:** We cannot simply increase n to 10 or 20 to fix this, because of the sparsity problem—we would never find those exact 20-word sequences in our training data to count them.

```
"today the price of gold per ton , while  
production of shoe lasts and shoe  
industry , the bank intervened just  
after it considered and rejected an imf  
demand to rebuild depleted european  
stocks , sept 30 end primary 76 cts a  
share ."
```

— Trigram Model Output

Evaluating Models: Perplexity

How do we quantify if one language model is better than another? The standard evaluation metric in NLP is **Perplexity**.

Perplexity measures how "surprised" a model is by a real, unseen dataset. It is the inverse probability of the text normalized by the number of words.

$$\begin{aligned} \text{PPL} (W) &= P (w_1 , w_2 , \dots , w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\prod_{i=1}^N \frac{1}{P (w_i \mid w_1 , \dots , w_{i-1})}} \end{aligned}$$

Rule of Thumb: A lower perplexity means the model assigned a higher probability to the actual human text, indicating it is a superior model. (Perplexity is mathematically equal to the exponential of the cross-entropy loss).

Building a Neural Language Model

How do we transition from brittle, count-based statistics to the dense representations of deep learning?

The Fixed-Window Neural LM

The first logical step in deep learning for LMs is to replace exact word matching with a neural network that evaluates a fixed window of **word embeddings**.



A Familiar Limitation

Notice the structural similarity to the n-gram model. This neural network **still discards context** that falls outside its predetermined window size (e.g., discarding the crucial context of "proctor started the clock").

Architecture of the Fixed-Window LM

Unlike n-grams, this model uses dense vectors to understand conceptual similarities between words, elegantly solving the sparsity problem.

1. Convert words to one-hot vectors.
2. Multiply by an embedding matrix E to get word embeddings $e^{\{t\}}$.
3. Concatenate the embeddings into a single vector e .
4. Pass through a hidden layer with weights W and bias b_1 , applying non-linearity.
5. Multiply by output weights U and apply Softmax for a probability distribution $\hat{y}$.

output distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

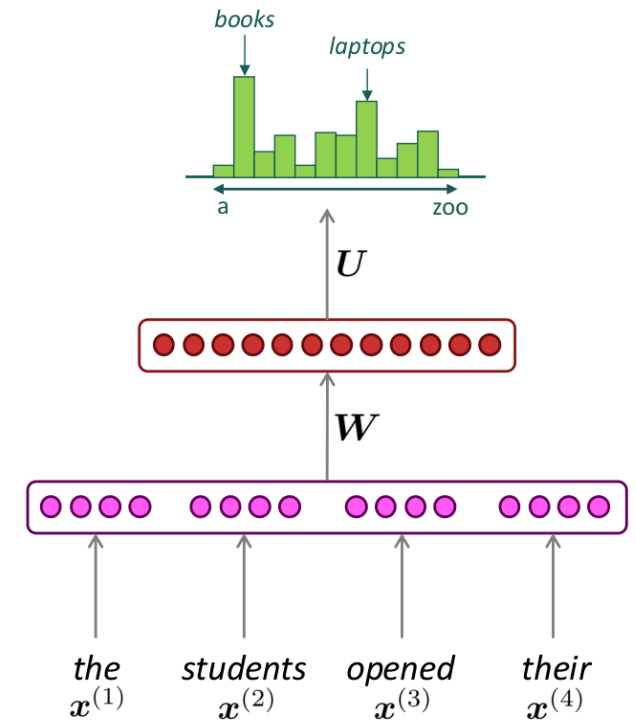
hidden layer

$$h = f(We + b_1)$$

concatenated word embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors
 $x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$



Evaluating the Fixed-Window LM

✓ Improvements

- **No Sparsity:** Uses dense embeddings, meaning it can generalize to unseen n-grams based on word similarity.
- **Less Storage:** We no longer need to store enormous count tables for every possible word combination in the English language.

✗ Remaining Flaws

- **Small Window:** Context size is strictly limited. Expanding the window explodes the weight matrix size W^2 .
- **Asymmetric Weights:** If a word appears at position 1 vs position 2, it is multiplied by entirely different weights. It fails to learn positional independence.

Conclusion: We need an architecture designed specifically for variable-length sequences.

Enter the Recurrent Neural Network

RNNs revolutionized NLP by introducing the concept of a "memory state".

The Core Innovation

Instead of processing a whole window simultaneously with unique weights for each position, an RNN applies the **exact same weights** W repeatedly, one word at a time, updating a continuous hidden state h that represents the entire history of the sequence.

Math of a Simple RNN Language Model

This is the "unrolled" view of the RNN computation over time.

At each timestep t , the network calculates a new hidden state by merging the previous hidden state (its memory) and the current word embedding.

HIDDEN STATE UPDATE

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

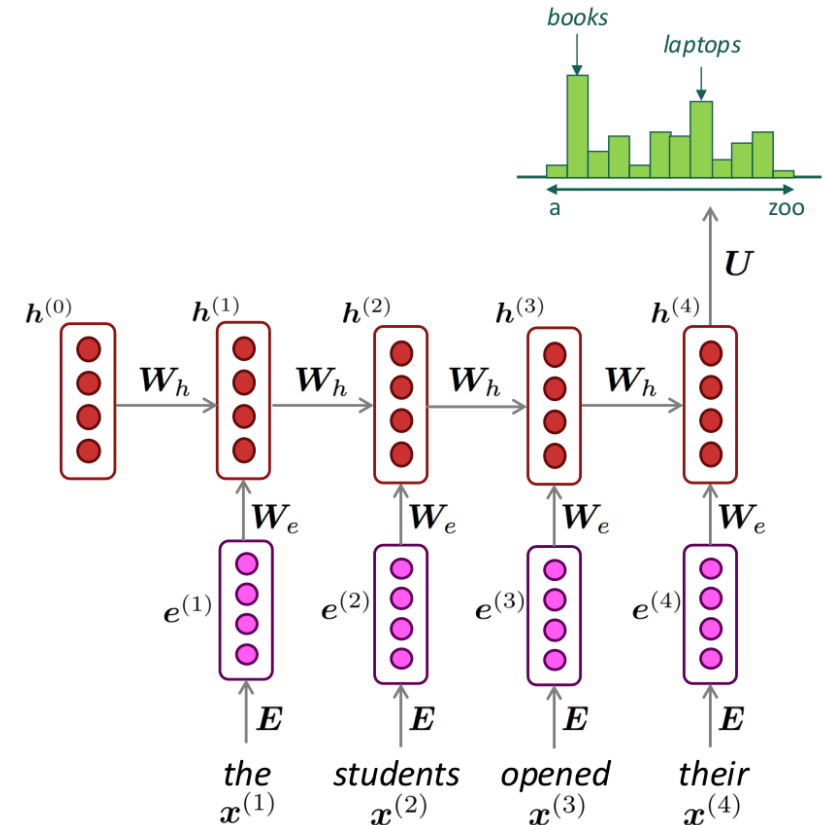
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Pros & Cons of RNN Language Models

+ Advantages

- **Infinite Context:** Theoretically handles sequences of any arbitrary length.
- **Consistent Model Size:** Because W_h is shared across all timesteps, the matrix doesn't grow as the input sequence gets longer.
- **Symmetry:** The exact same transformation is applied regardless of a word's position in the sentence.

- Disadvantages

- **Sequential Bottleneck:** Computation is fundamentally slow. Step t cannot begin until step $t-1$ finishes, preventing massive parallelization on GPUs.
- **Long-Term Memory:** In practice, despite the theoretical infinite context, standard RNNs struggle to retain information from many steps ago due to vanishing gradients.

Training an RNN Language Model

Training involves feeding a massive text corpus into the model. At every *single timestep*, the model computes an output distribution predicting the next word.

Cross-Entropy Loss

We calculate the cross-entropy loss between the model's predicted probability distribution $\hat{y}^{(t)}$ and the true next word $y^{(t)}$ (represented as a one-hot vector).

The total loss over the entire sequence is simply the average of these individual losses at each timestep.

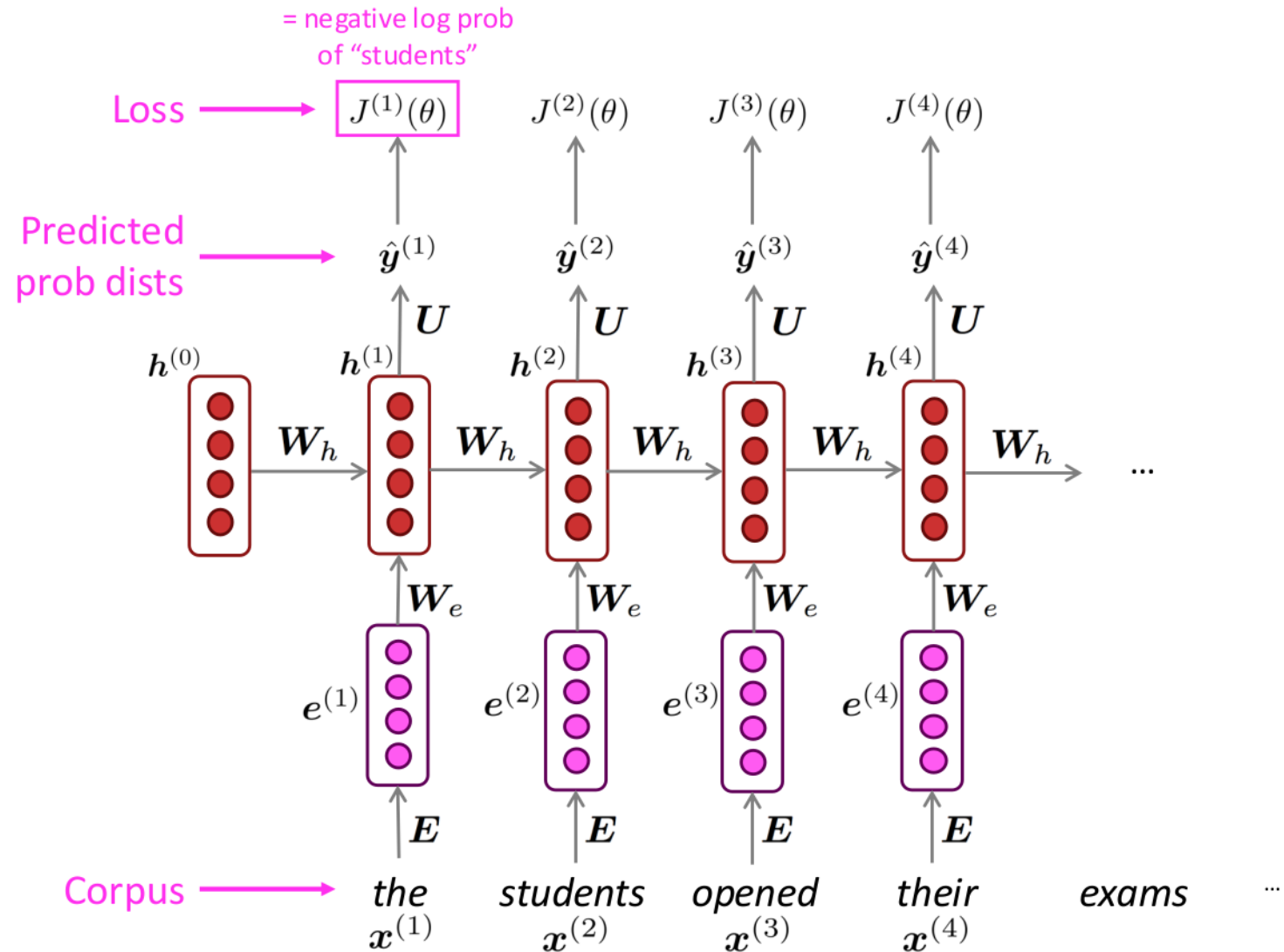
$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

Average loss over
each timestep

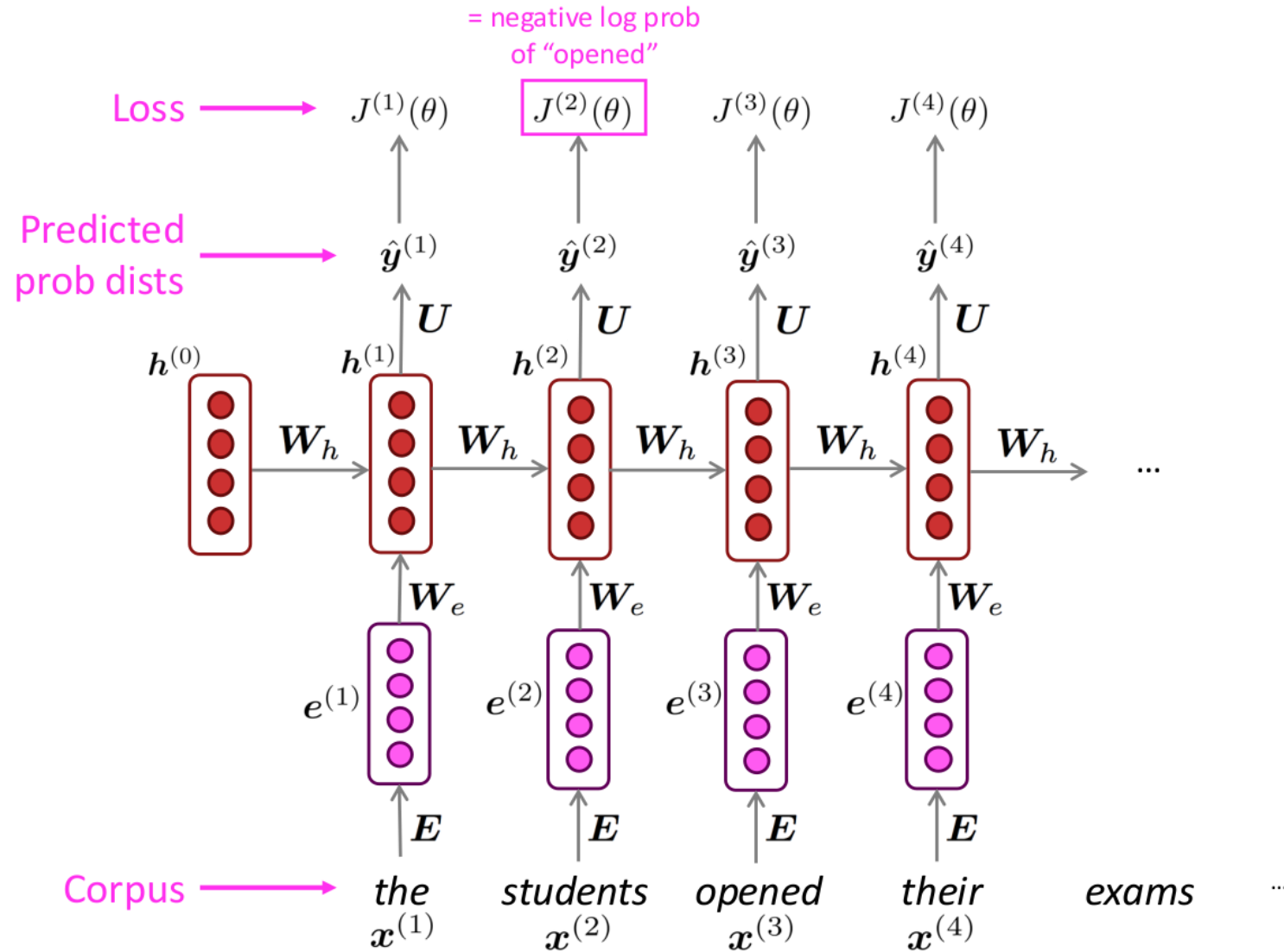


$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

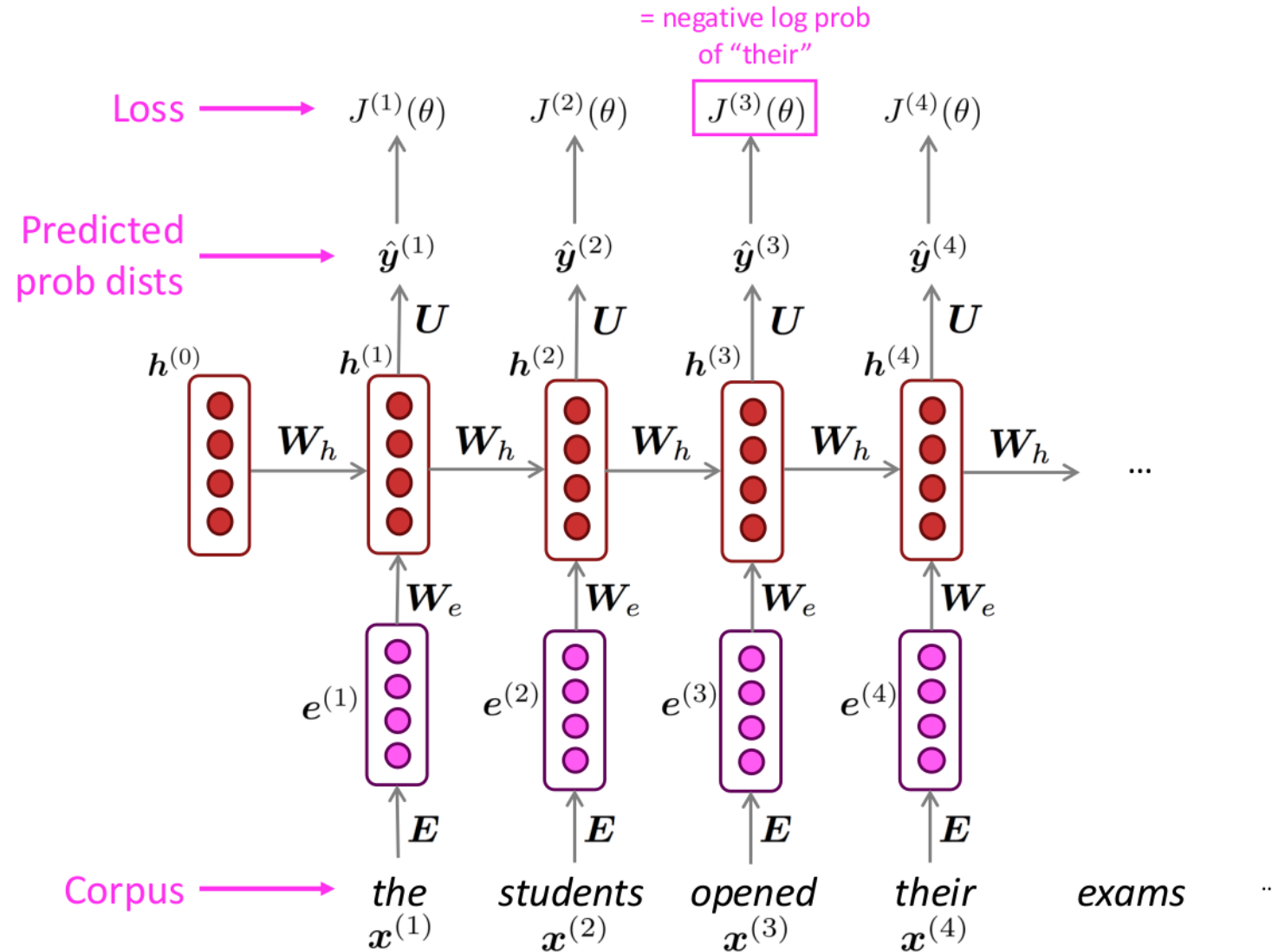
Training Visualization: Timestep 1



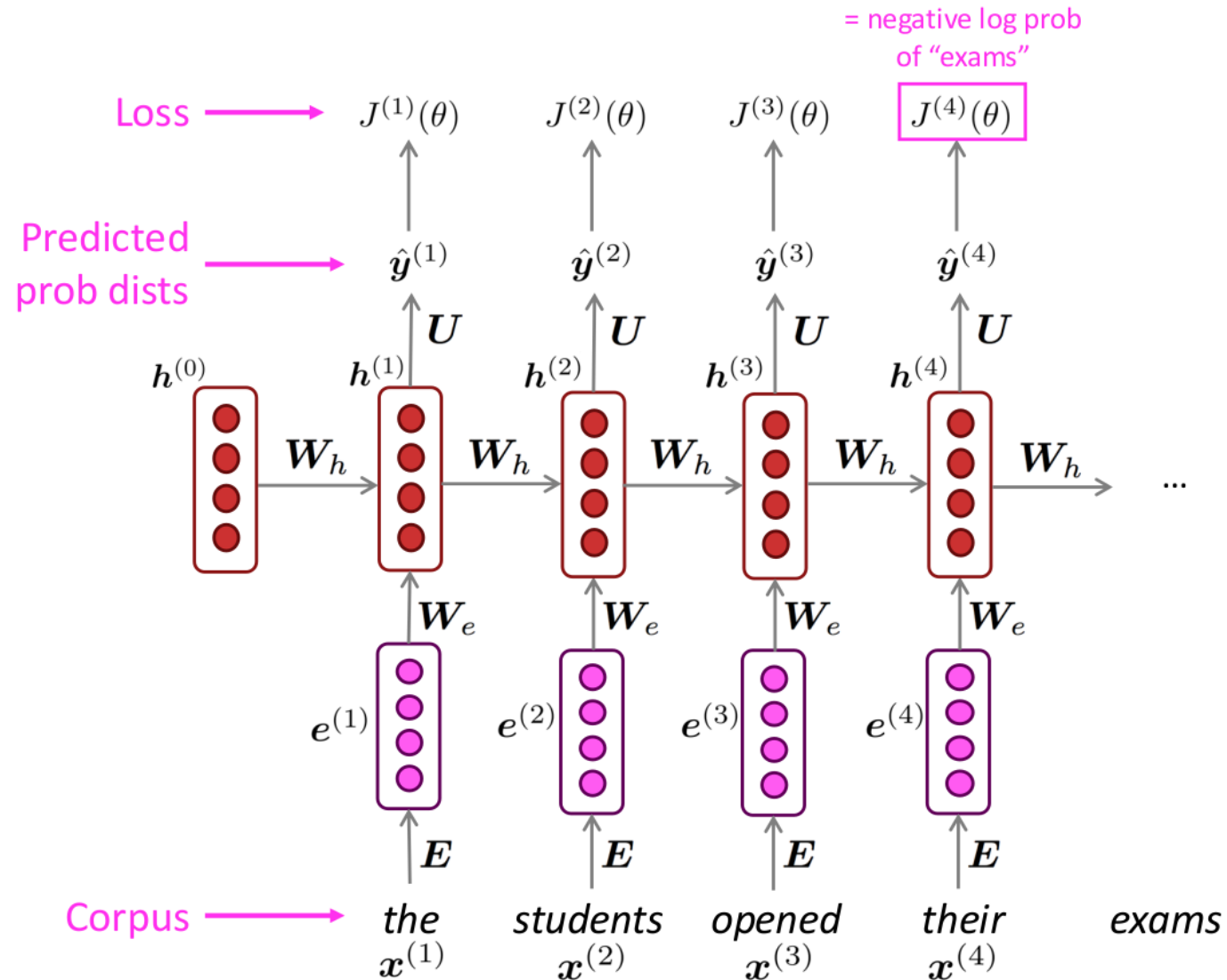
Training Visualization: Timestep 2



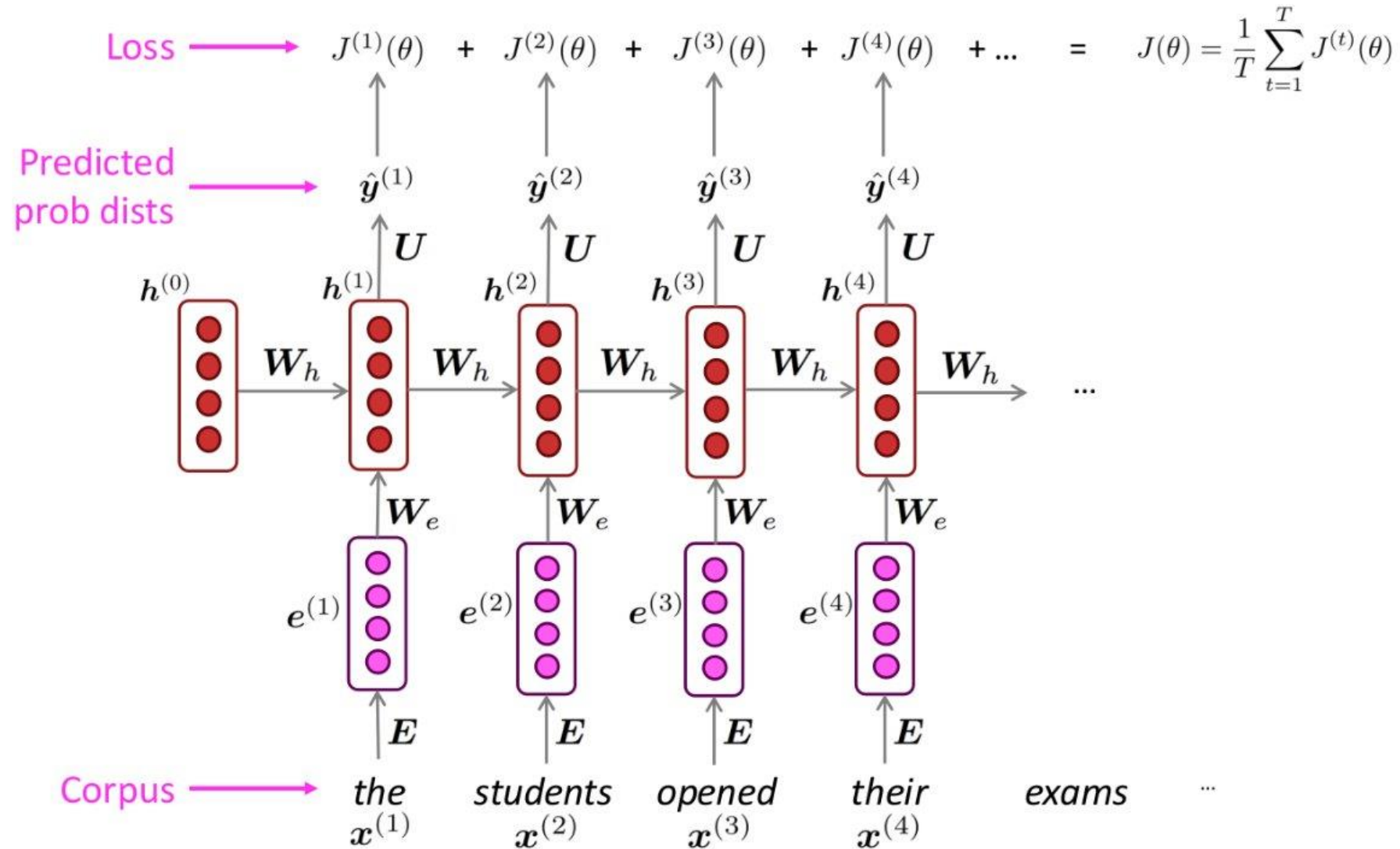
Training Visualization: Timestep 3



Training Visualization: Timestep 4



Overall Loss & Teacher Forcing



Overall Loss & Teacher Forcing

Averaging Individual Losses

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

The total sequence loss is the mean of individual timestep losses.

Teacher Forcing Explained

To keep training stable, we use **Teacher Forcing**.

Regardless of whether the model predicted the correct word at $t-1$, we always feed the **actual ground-truth word** into the next timestep t .

This prevents "error drift" where a single early mistake causes the entire sequence generation to spiral into nonsense during training.

Memory, Batches, and Efficiency



Memory Constraints

Computing gradients across an entire massive corpus (like all of Wikipedia) at once is impossible. We would run out of GPU memory instantly.



Batching Strategy

We chop the text into **batches** (e.g., individual sentences or fixed-length sequences) of size B . This allows for parallelized computation.



SGD Updates

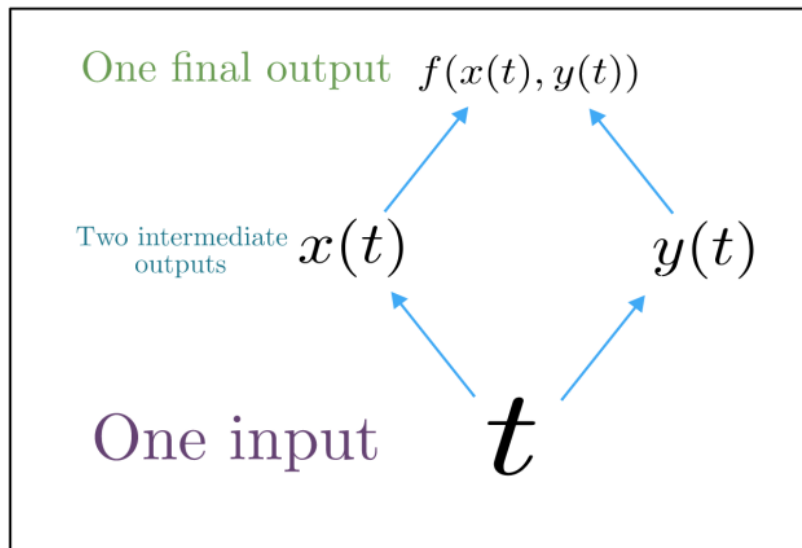
We apply Stochastic Gradient Descent (SGD) to update weights after each batch. This provides a "noisy" but highly efficient path to convergence.

Multi-Variable Chain Rule

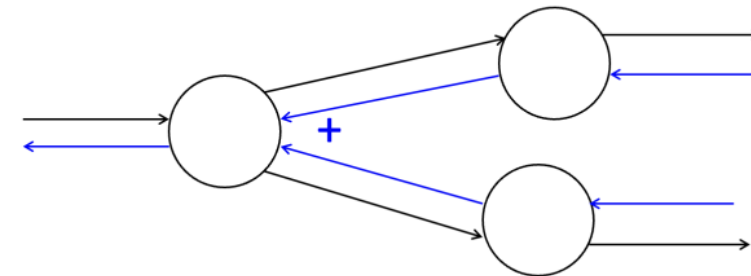
- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

Derivative of composition function



Gradients sum at outward branches



$$a = x + y$$

$$b = \max(y, z)$$

$$f = ab$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \frac{\partial a}{\partial y} + \frac{\partial f}{\partial b} \frac{\partial b}{\partial y}$$

Unpacking BPTT Machinery

Backpropagation Through Time (BPTT) is the fundamental engine used to train Recurrent Neural Networks. Unlike standard Feed-Forward networks, RNNs apply the **exact same weight matrix** W_h at every single timestep.

The core mathematical challenge is determining how a single weight matrix impacts the loss J across a sequential chain of dependencies. We represent the total gradient as the sum of localized gradients computed at each unrolled timestep i .

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial W_h | (i)}$$

The term on the right asks: "How much does the loss change if we tweak W_h only at timestep i , while holding all other applications of W_h mathematically constant?"

This allows us to isolate the error contribution of a specific moment in the sequence before aggregating them for the final weight update.

Full Computation Example (T=4)

Step	Chain Rule Derivation for Local Gradient
i = 4	$(\frac{\partial J}{\partial h^{(4)}}) \times \frac{\partial h^{(4)}}{\partial W_h}$
i = 3	$(\frac{\partial J}{\partial h^{(4)}} \cdot \frac{\partial h^{(4)}}{\partial h^{(3)}}) \times \frac{\partial h^{(3)}}{\partial W_h}$
i = 2	$(\frac{\partial J}{\partial h^{(4)}} \cdot \frac{\partial h^{(4)}}{\partial h^{(3)}} \cdot \frac{\partial h^{(3)}}{\partial h^{(2)}}) \times \frac{\partial h^{(2)}}{\partial W_h}$
i = 1	$(\frac{\partial J}{\partial h^{(4)}} \cdot \frac{\partial h^{(4)}}{\partial h^{(3)}} \cdot \frac{\partial h^{(3)}}{\partial h^{(2)}} \cdot \frac{\partial h^{(2)}}{\partial h^{(1)}}) \times \frac{\partial h^{(1)}}{\partial W_h}$



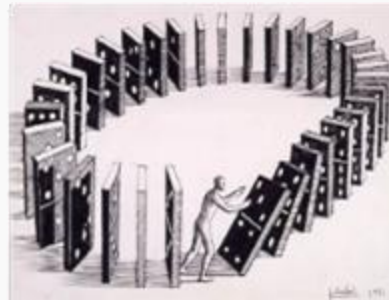
Final Gradient = Step 1 + Step 2 + Step 3 + Step 4

The Domino Chain Rule

Hidden states in an RNN form a strict sequential dependency. To find how a change in the first memory state affects the final loss, we multiply partial derivatives backward—a mathematical "**Domino Effect**."



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial J^{(4)}}{\partial h^{(4)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(2)}}{\partial h^{(1)}}$$



Unique Variable Pretense

The Concept

Momentarily pretend that the weight matrix used at each step is a unique variable. This decoupling allows us to apply standard partial differentiation logic to each instance.

The Notation

We define unique indices for each temporal step:

$W_{h,3}$ for Step 3

$W_{h,4}$ for Step 4

Note: They hold the exact same values but are treated as distinct inputs.

Loss Function Dependencies

- 🔗 **Primary Dependency:** The final loss $J^{(4)}$ depends strictly on the final hidden state $h^{(4)}$.
- 🔗 **Direct Input Chain:** Expanding the state shows that only $W_{h,4}$ and $h^{(3)}$ appear in the equation.
- 🚫 **The Missing Weight:** Notice that $W_{h,3}$ does **not** appear anywhere in the top-level direct equations.

$$h^{(4)} = \sigma (W_{h,4} h^{(3)} + W_x x^{(4)} + b)$$

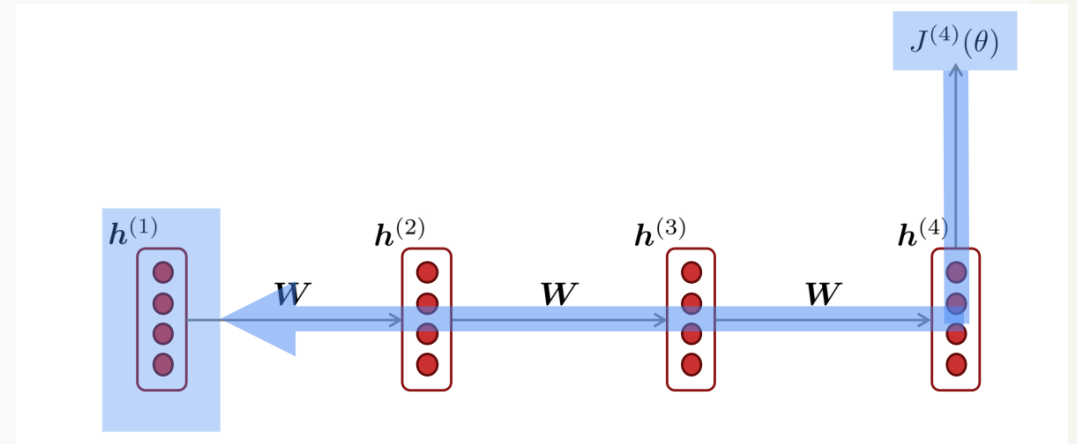
The Hidden Variable

Hidden Inside $h(3)$

The variable $W_{h,3}$ is "hidden" entirely inside the computation of the previous state $h^{(3)}$:

$$h^{(3)} = \sigma (W_{h,3} h^{(2)} + W_x x^{(3)} + b)$$

Because there is no direct link between the loss and step 3 weights, we cannot perform a direct derivative. We are forced to bridge this mathematical gap.



Bridging with the Chain Rule

STEP 3 LOCALIZED GRADIENT

$$\frac{\partial J^{(4)}}{\partial W_h} \mid (3)$$

The Bridge

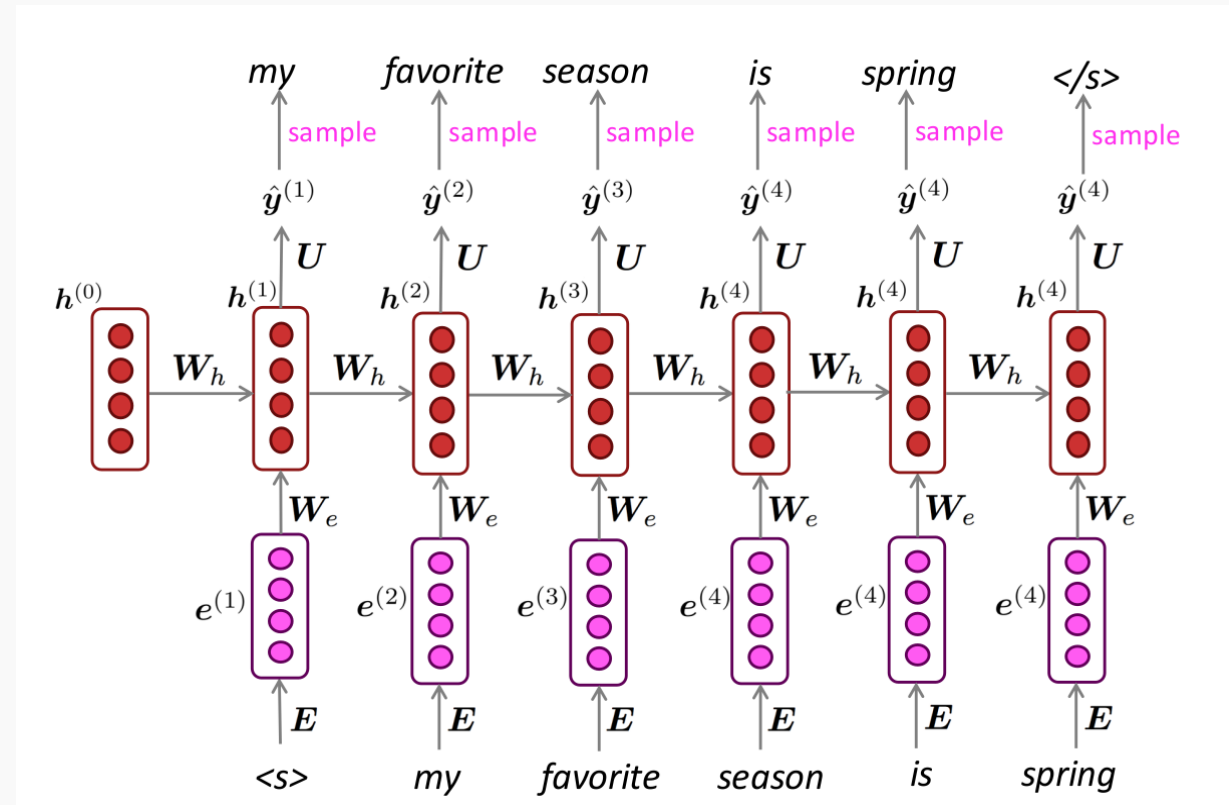
We multiply how the loss changes with respect to the output of step 3, by how step 3 changes with respect to its own weights:

$$\frac{\partial J^{(4)}}{\partial h^{(3)}} \times \frac{\partial h^{(3)}}{\partial W_{h,3}}$$

Generating Text with RNNs

Generating text follows the same "auto-regressive" logic as n-grams, but with the power of infinite memory potential.

- 1 Start with a "start" token or initial context.
- 2 The network outputs a probability distribution.
- 3 **Sample** a word from that distribution.
- 4 Feed that chosen word **back into the network** as the input for the next step.



Fun Example Harry Potter RNN

"The common room was a large, stone room with a fireplace and a large, wooden table. Harry looked around. The common room was perched on a shoulder..."

— Generated by a vanilla RNN trained on J.K. Rowling's books

Analysis: The grammar is remarkably consistent, but the narrative coherence is weak. Why would a room be "perched on a shoulder"? This illustrates the limit of vanilla RNNs in capturing deep semantic logic.

Problem with Vanilla RNN

Vanishing and Exploding Gradient

Vanishing Gradient Intuition

The Core Question

How much does an error made at the very end of a sentence ($J(4)$) tell us about how we should adjust the memory from the very beginning ($h(1)$)?

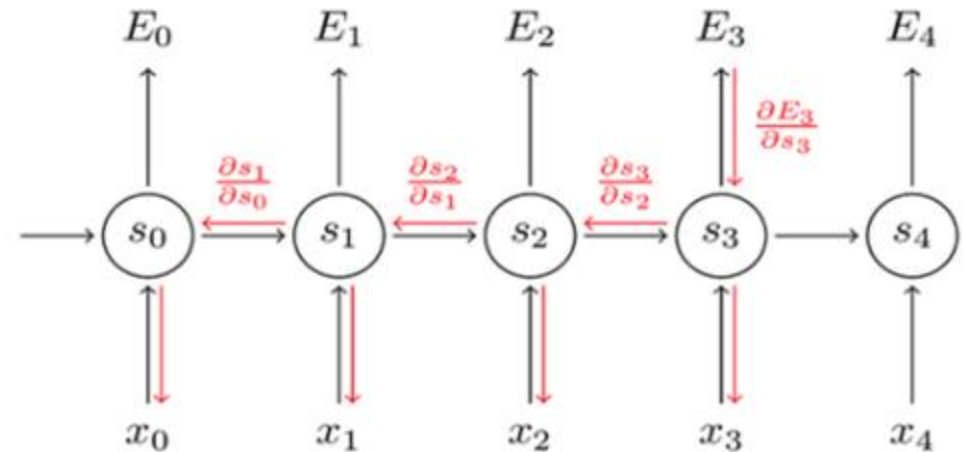
$$\frac{\partial J^{(4)}}{\partial h^{(1)}}$$

This derivative is the mathematical measure of that influence.

Chain Rule: the First Step Back

To find the error signal at step 1, we must first pass it through step 2. We use the chain rule to link the final loss to $h(1)$ via the intermediate hidden states.

$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$



Backpropagation Through Time

Expanding the Chain Backward

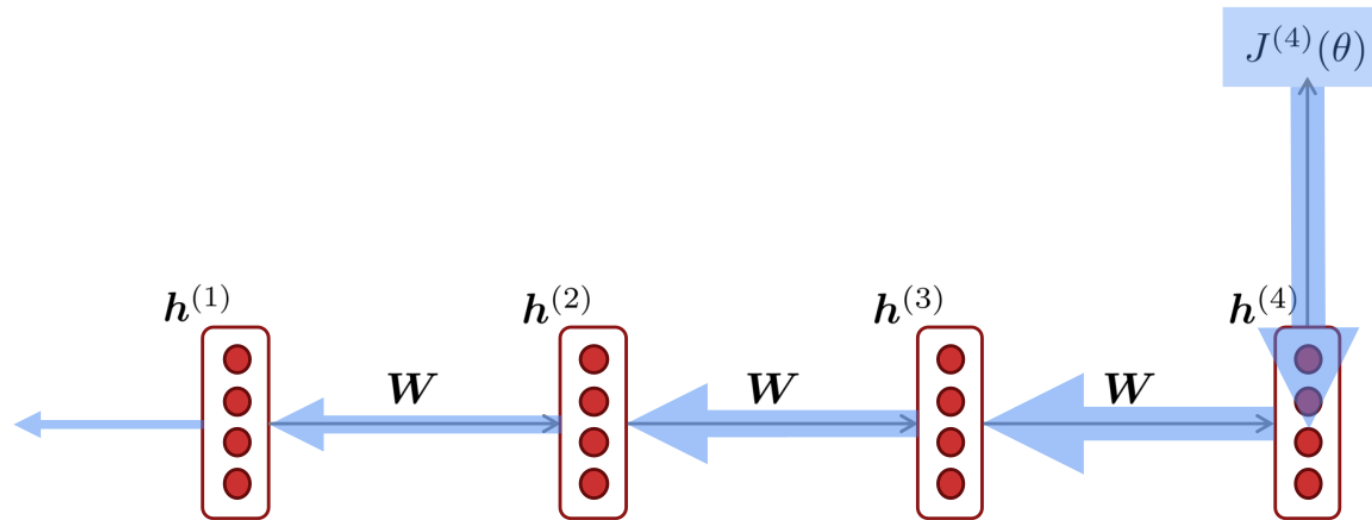
The error signal continues its journey backward through the hidden states. We add more terms to the chain, moving from $h(3)$ to $h(2)$ and finally back towards the beginning.

Notice that we are essentially multiplying Jacobian matrices of the hidden state transition at each step.

$$\frac{\partial h^{(3)}}{\partial h^{(2)}}$$

This term represents the influence of the memory at step 2 on the memory at step 3.

The Full Multiplication Chain



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \boxed{\frac{\partial h^{(2)}}{\partial h^{(1)}}} \times \boxed{\frac{\partial h^{(3)}}{\partial h^{(2)}}} \times \boxed{\frac{\partial h^{(4)}}{\partial h^{(3)}}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem:
When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Vanishing Gradient Intuition



The Chain Rule Problem

During backpropagation, the gradient at timestep t is calculated by repeatedly applying the chain rule through intermediate timesteps. If these partial derivatives are small (e.g., less than 1), they multiply exponentially.



Signal Decay

Multiplying a long sequence of fractions causes the resulting product to shrink towards zero. Consequently, the gradient signal "vanishes" before it can ever reach the initial words of a long sequence, effectively erasing their influence on training.

| The Role of Eigenvalues

Largest Eigenvalue

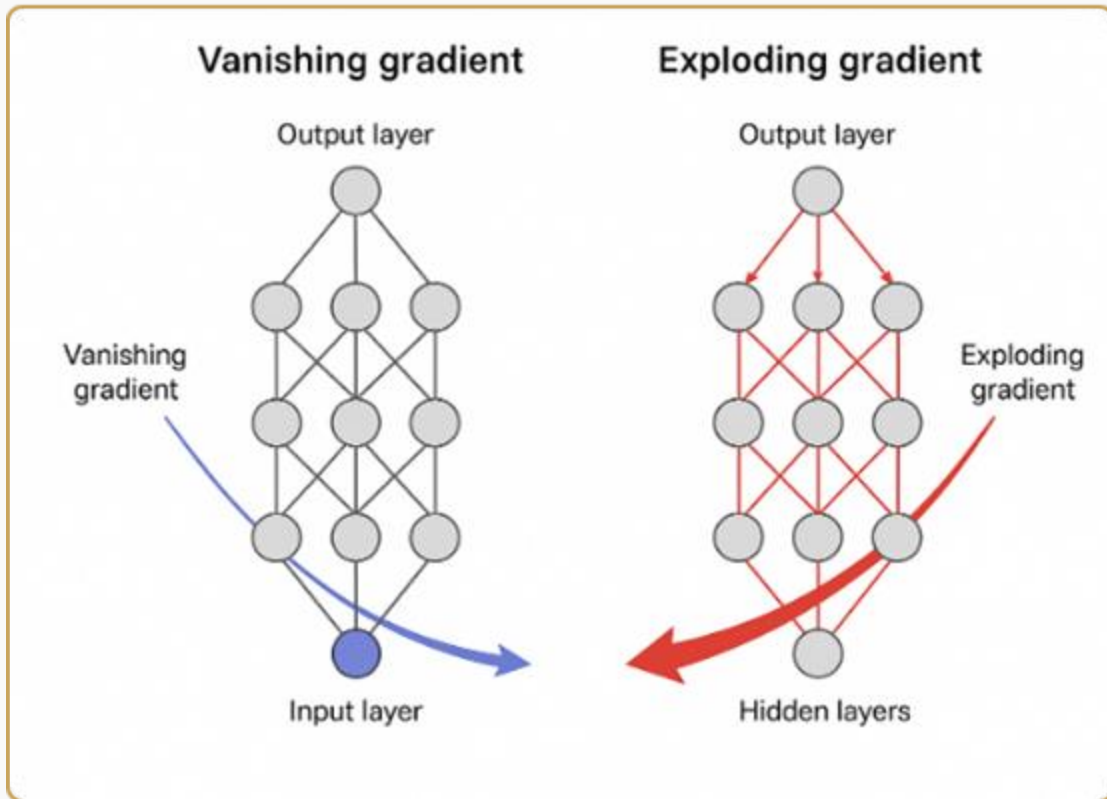
$$\lambda < 1$$

Spectral Radius and Convergence

From linear algebra, we know that if the recurrent weight matrix **W** is diagonalizable, raising it to the power l corresponds to raising its eigenvalues to that same power.

If the **largest eigenvalue (spectral radius)** is less than 1, then the term **W^l** will approach the zero matrix exponentially fast as the distance l increases. This is the formal mathematical cause for the RNN's inability to maintain long-term memory.

Visualizing Gradient Magnitude



This visualization highlights the stark contrast between gradient signals based on their distance from the loss calculation:

- ⚠ **Near Effects:** The gradient signal is robust, allowing the model to easily learn local syntactic dependencies.
- 👤 **Faraway Effects:** The signal vanishes to nearly zero. The network physically cannot receive feedback about how to adjust weights to account for words that appeared much earlier in the sequence.

Result: Weights are updated based only on short-term context.

Example: Long-Distance Dependency

*"When she tried to print her **tickets**, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____"*

The Vanishing Signal

To correctly predict the target word "**tickets**", the model must connect it to the mention 7 steps ago. In a vanilla RNN, the gradient flow from the target back to the first sentence vanishes. Consequently, the model cannot "see" the relationship and generates inconsistent or generic text.

Solving Vanishing Gradient

vanishing gradients requires **structural innovations** in the neural architecture itself.



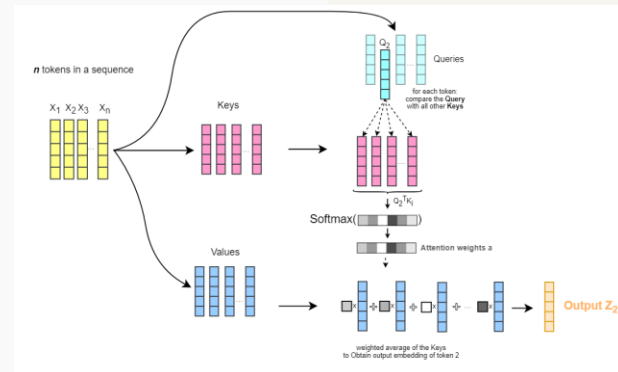
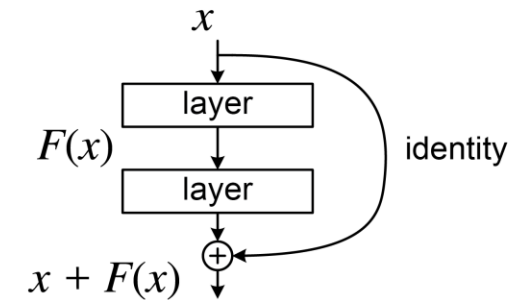
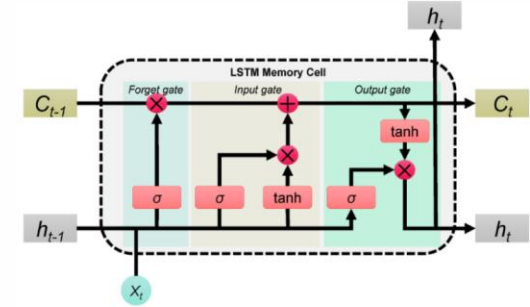
Separate Memory: Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) use specialized "gates" to decide what to preserve.



Skip Connections: Residual networks (ResNets) allow the signal to bypass layers.



Attention: Allows the model to look directly at the past, bypassing the sequential chain entirely.



The Exploding Gradient Problem



Instability

If the largest eigenvalue of $\mathbf{Wh}$ is greater than 1, raising the matrix to a power causes gradients to grow exponentially. This results in massive, unstable updates during SGD.



Numerical Failure

The weights are updated by values so large that the model's parameters enter unusable regions of the space. This often results in **Inf** or **NaN** (Not a Number) values appearing in your training logs, forcing the process to crash.

Solution Gradient Clipping

- **Gradient clipping**: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

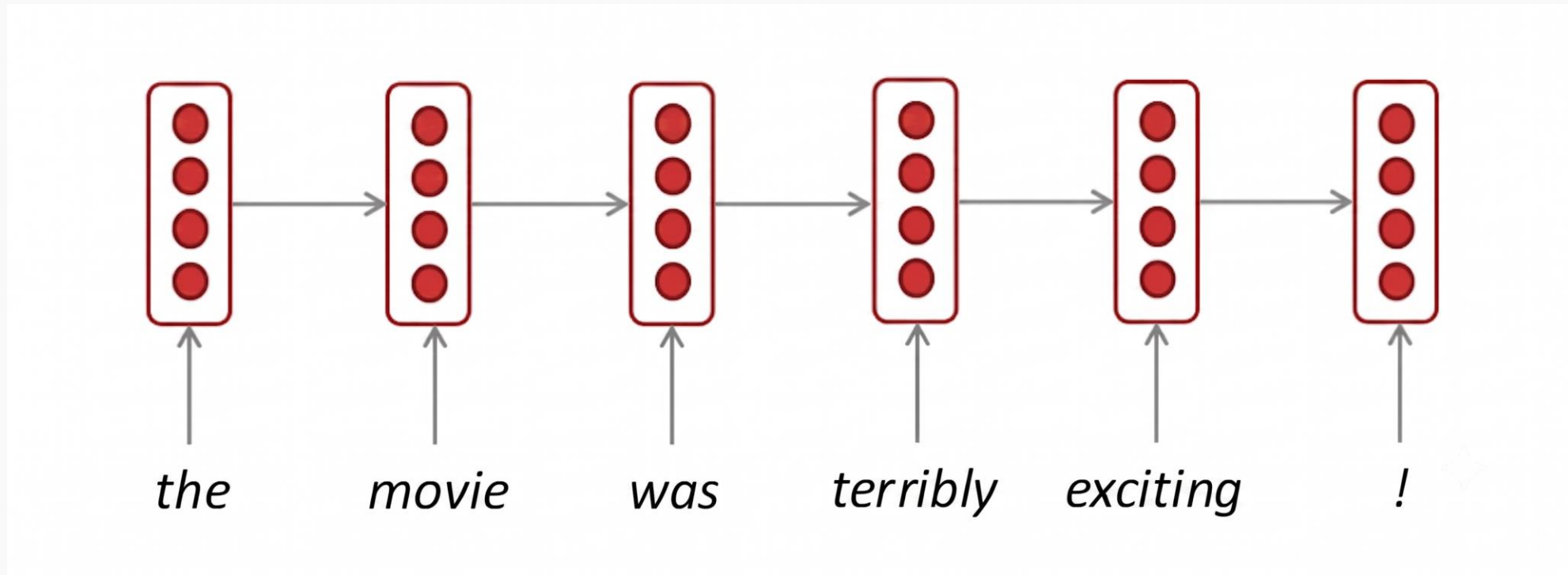
$$\begin{aligned} \hat{\mathbf{g}} &\leftarrow \frac{\partial \mathcal{E}}{\partial \theta} \\ \text{if } \|\hat{\mathbf{g}}\| &\geq \textit{threshold} \text{ then} \\ &\quad \hat{\mathbf{g}} \leftarrow \frac{\textit{threshold}}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}} \\ \text{end if} \end{aligned}$$

- **Intuition**: take a step in the same direction, but a smaller step
- In practice, **remembering to clip gradients is important**, but exploding gradients are an easy problem to solve

RNN Extensions

Expanding On RNN Architecture

Why Left-to-Right Fails



In this sentence, "terribly" is a positive amplifier. However, a left-to-right RNN would process "terribly" as **negative** because it hasn't seen the word "exciting" yet.

Context from the future is often required to understand the past.

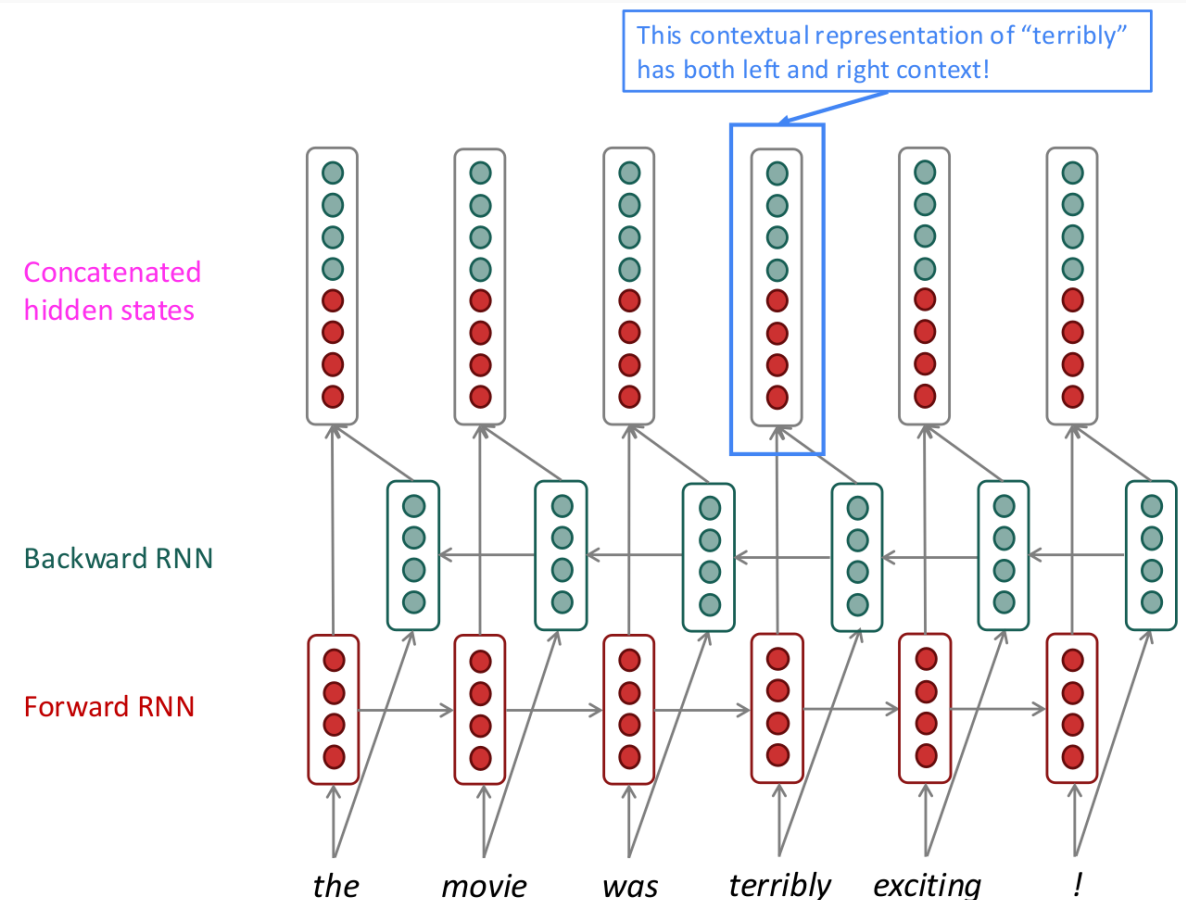
Bidirectional RNNs

Bidirectional RNNs

Run two networks simultaneously:

- **Forward:** the → movie → was → terribly
- **Backward:** terribly ← exciting ← !

States from both are **concatenated**, providing a complete understanding of every word.

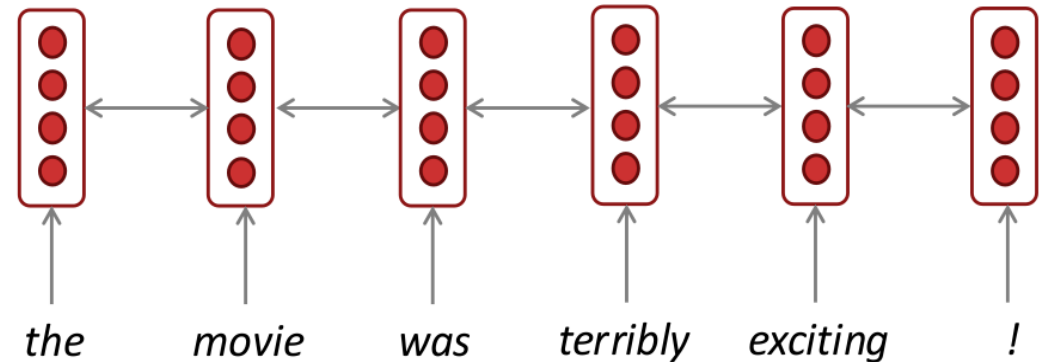


Formal Bidirectional Notation

This formalizes the bidirectional paradigm. We process the sequence using two distinct RNNs: a **Forward RNN** and a **Backward RNN**. Each network maintains its own specific weight matrices optimized for its respective direction.

$$\begin{aligned}\vec{h}^{(t)} &= \text{RNN}_{\text{fw}}(\vec{h}^{(t-1)}, x^{(t)}) \\ \overleftarrow{h}^{(t)} &= \text{RNN}_{\text{bw}}(\overleftarrow{h}^{(t+1)}, x^{(t)})\end{aligned}$$

The final hidden state passed to the subsequent layers is the concatenated vector, "stapling" the two meaning representations together into one dense feature set.



RNNs Logical Applicability Constraints

Forbidden for LM

Bidirectionality is **not applicable** to standard Language Modeling. In a task like predictive typing, you cannot look into the "future" because those words are exactly what you are trying to guess. The backward network would have access to the target labels during training.

Ideal for Encoding

It is, however, extremely powerful for **Encoding** tasks where the entire input sequence is fixed and known at the start. **BERT** is the most famous example of a bidirectional context system that uses this "whole-sentence" knowledge to understand nuance.

Stacked Multi-layer RNNS



Hierarchical Depth

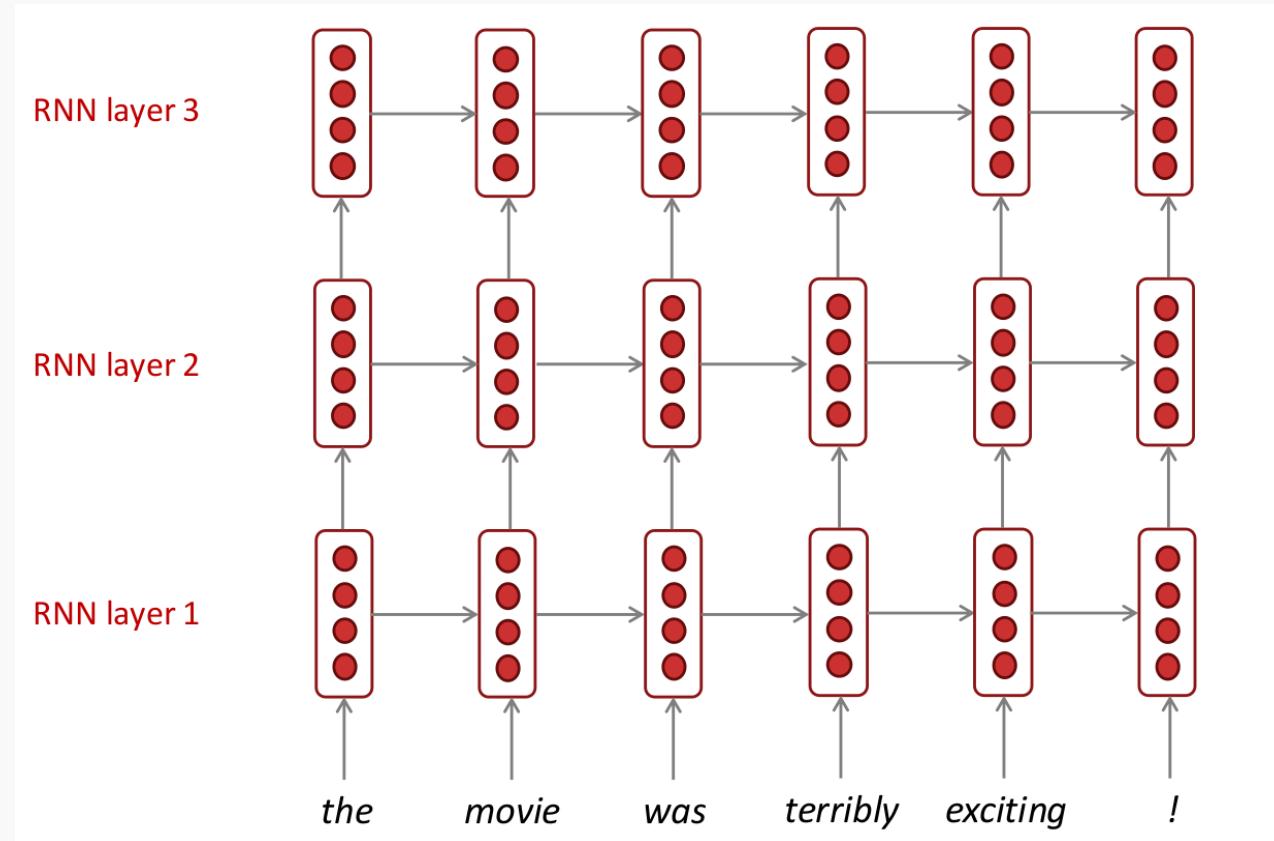
Vanilla RNNs are "deep" in time (horizontally), but we can also make them deep **spatially** (vertically). This stacking allows for a hierarchy of features, much like the layers of an ice cream cone.

- **Lower RNNs:** Compute local, concrete features like word syntax or part-of-speech.
- **Higher RNNs:** Compute abstract semantics, global themes, and stylistic tone.

3-Layer Architecture Flow

Information flows vertically through the network. At each timestep, the hidden state output of a lower layer becomes the input for the layer above it.

- **Layer 1:** Processes the raw word embeddings directly.
- **Layer 2:** Processes the "processed thoughts" emitted by the first layer.
- **Layer 3:** Synthesizes the highest-level abstract representations of the entire sentence structure.



Multi-layer RNN Engineering Practice



Optimizing Depth: Research by Britz et al. (2017) suggests that for Neural Machine Translation, 2 to 4 layers is the "sweet spot" for the encoder, while the decoder benefits from roughly 4 layers.



Instability Limits: Unlike convolutional networks, RNNs become mathematically unstable very quickly as you add vertical layers. Training an 8-layer RNN is incredibly difficult and strictly requires **skip-connections** (residuals).



Transformers vs RNNs: If your task requires massive depth (e.g., 12 or 24 layers), modern architectures like **Transformers** are fundamentally superior because they avoid the recurrent instability.

Machine Traslation

From cold War Dictionaries to deep Sequence learning

Machine Translation

Machine Translation (MT) is the task of translating a sentence x from one language (the **source language**) to a sentence y in another language (the **target language**).

$x:$ *L'homme est né libre, et partout il est dans les fers*



$y:$ *Man is born free, but everywhere he is in chains*

– Rousseau

1950s: Dawn of Translation

Military Funding

Heavy Cold War funding from the military aimed to automate the surveillance of Russian documents into English.

Intractable Problems

Ambiguity, syntax, and semantics quickly made the problem appear unsolvable to early computer scientists.

Era of Calculators

Research began on machines that were mathematically less powerful than a modern pocket calculator.

Rule-Based Systems

Early attempts were simple rule-based word substitution systems (dictionary-lookup scripts).

1990-2010 SMT: Statistical Approach

- Core idea: Learn a **probabilistic model** from **data**
- Suppose we're translating French $\rightarrow$ English.
- We want to find **best English sentence** y , given **French sentence** x

$$\operatorname{argmax}_y P(y|x)$$

- Use Bayes Rule to break this down into **two components** to be learned separately:

$$= \operatorname{argmax}_y \underbrace{P(x|y)}_{\text{Translation Model}} \underbrace{P(y)}_{\text{Language Model}}$$

Translation Model

Models how words and phrases should be translated (*fidelity*).
Learned from parallel data.

Language Model

Models how to write good English (*fluency*).
Learned from monolingual data.

The Complexity of Statistical MT



Engineering Nightmare

Systems were incredibly complex pipelines with hundreds of fine-tuned sub-modules for alignment and reordering.



Manual Feature Labors

Required specialized linguistic teams to manually engineer features for every language pair to handle word-order shifts.



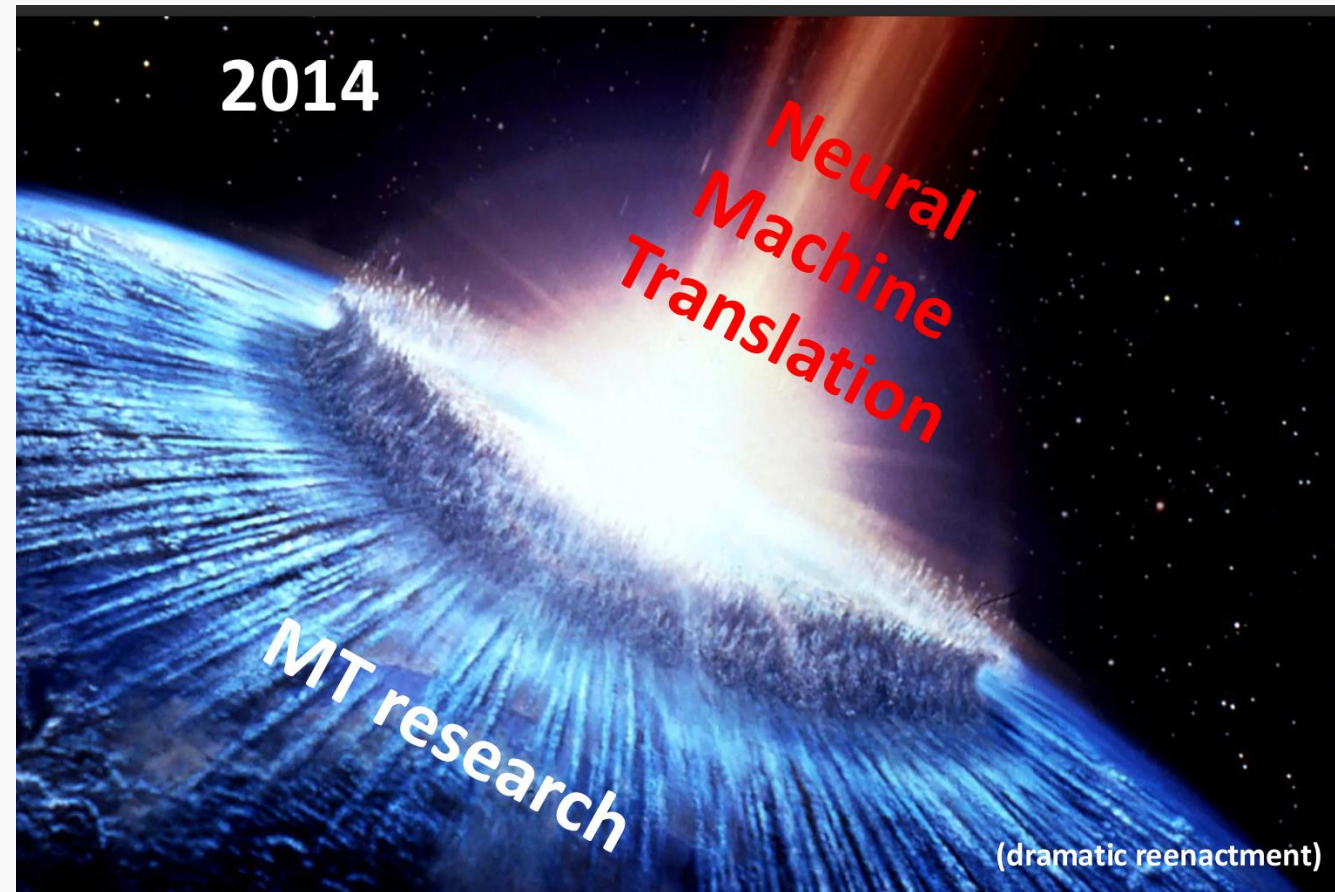
Phrase Tables

Demanded massive human effort to compile and prune phrase tables—essentially billions of rows of local translation rules.

2014: The NMT Paradigm Shift

In 2014, **Neural Machine Translation (NMT)** arrived and effectively "body-slammed" the traditional research field.

Deep learning proved that it could learn its own rules for translation from raw data, rendering decades of carefully tuned statistical methodology and linguistic feature engineering obsolete almost overnight.



NMT Success: Industry Adoption

2016

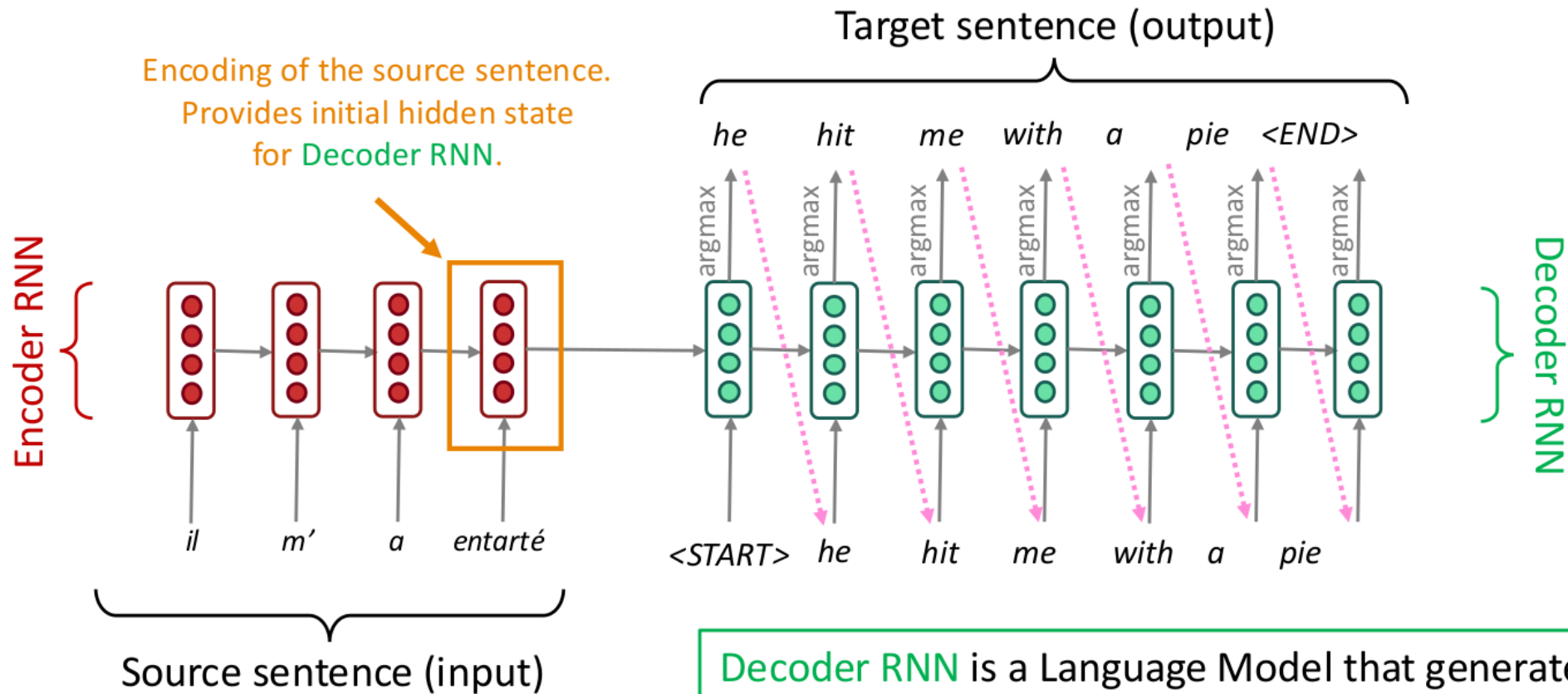
GOOGLE SWITCHED ENTIRELY



By 2016, the shift was absolute. Google Translate abandoned SMT for NMT, soon followed by Microsoft, Facebook, and Baidu.

- ✓ First seq2seq paper published in 2014.
- ✓ A small group of neural engineers replaced hundreds of linguistic engineers.
- ✓ Performance jumped more in months than it had in the previous decade.

The Unified Seq2Seq Model



Encoder RNN produces an **encoding** of the source sentence.

Decoder RNN is a Language Model that generates target sentence, *conditioned on encoding*.

Note: This diagram shows **test time** behavior: decoder output is fed in as next step's input

Versatility of Seq2Seq Framework



Summarization

Translating a long text sequence into a concise, shorter sequence.



Dialogue

Translating a previous utterance into the most helpful response.



Code Generation

Translating natural language intent into executable source code.

The ultimate realization: Almost everything in language is just a sequence-to-sequence translation task.

NMT: A Conditional Language Model

- The **sequence-to-sequence** model is an example of a **Conditional Language Model**
 - **Language Model** because the decoder is predicting the next word of the target sentence y
 - **Conditional** because its predictions are *also* conditioned on the source sentence x

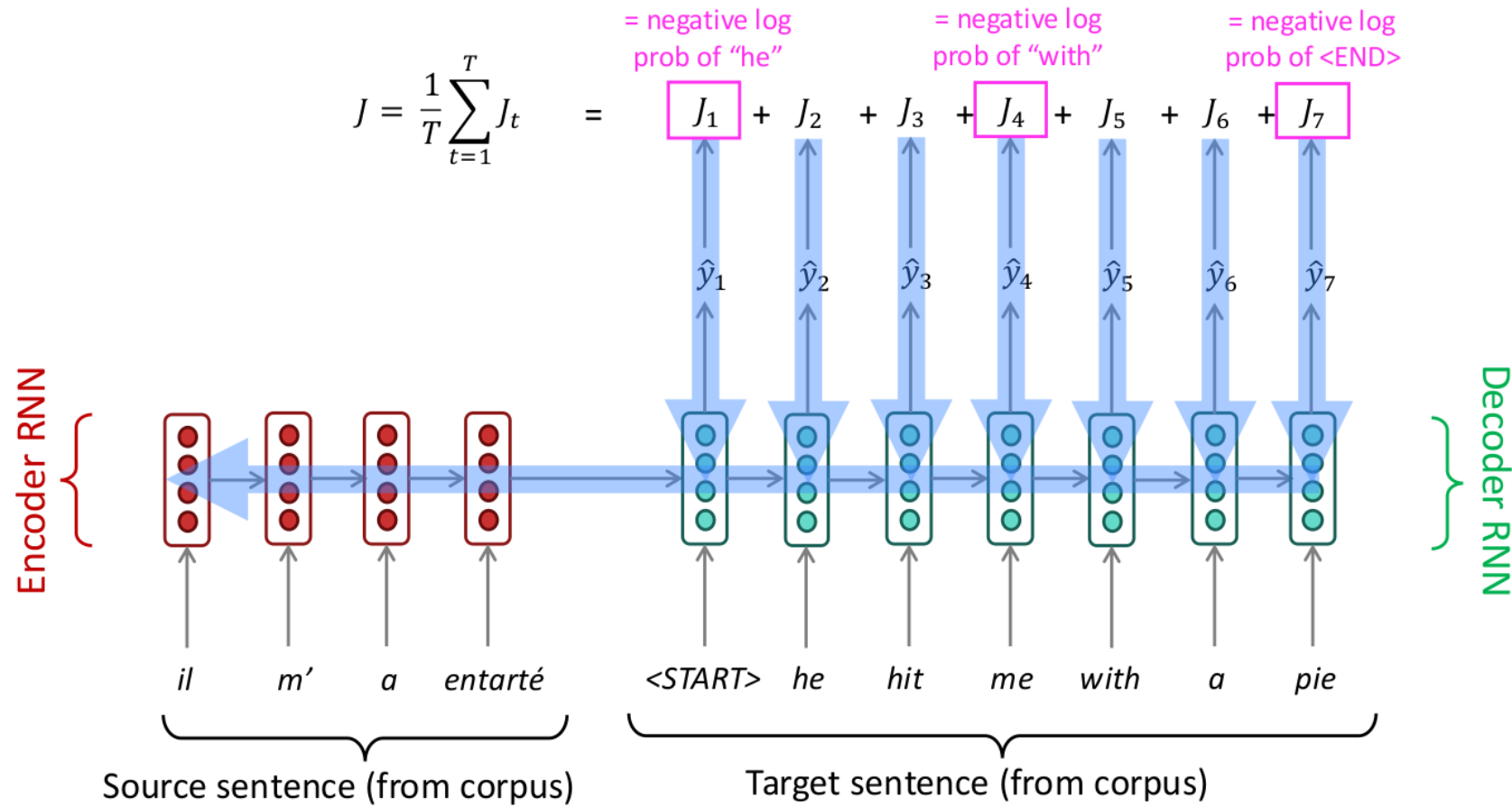
- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots \underbrace{P(y_T|y_1, \dots, y_{T-1}, x)}$$

Probability of next target word, given
target words so far and source sentence x

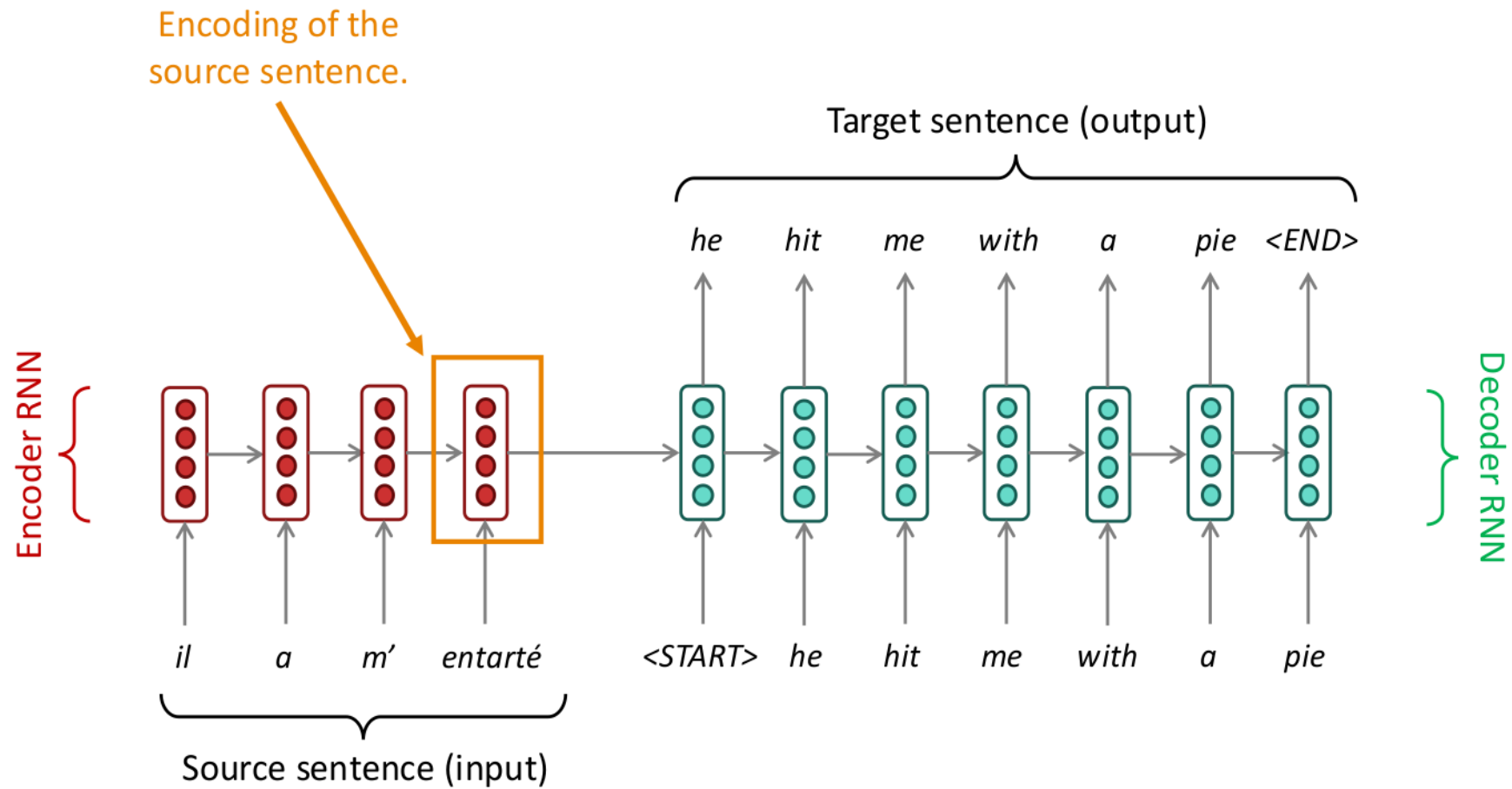
- **Question:** How to train an NMT system?
- **(Easy) Answer:** Get a big parallel corpus...
 - But there is now exciting work on “unsupervised NMT”, data augmentation, etc.

End-to-End NMT Training



Seq2seq is optimized as a **single system**. Backpropagation operates "*end-to-end*".

The Information Bottleneck



Problems with this architecture?

Linear Interaction Distance

$O(\text{Sequence Length})$

In recurrent models, information must travel through one step for every word in the sequence. Distant words struggle to communicate.

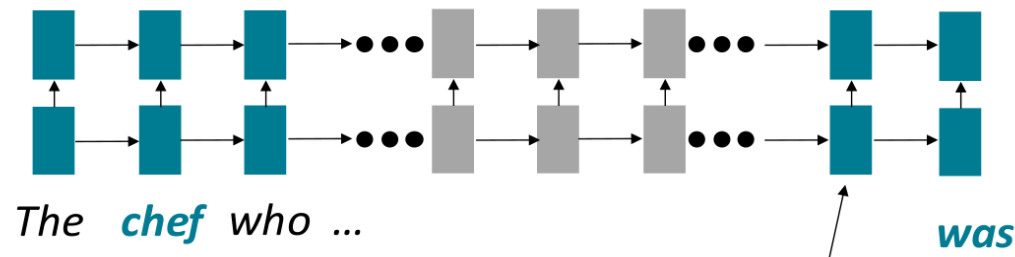
$O(n)$



Example Case

"The **chef** who cooked the meal for the large group of hungry travelers **was**..."

The model must propagate the plural/singular state through many matrix multiplications, often losing the subject before reaching the verb.



Info of **chef** has gone through $O(\text{sequence length})$ many layers!

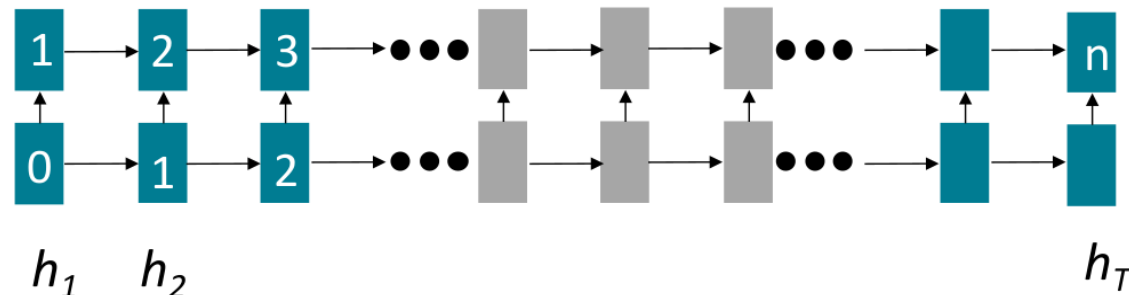
Lack of Parallelizability

Sequential

$O(N)$ STEPS

GPU Underutilization

RNNs are inherently sequential: step t requires the result of $t-1$. This prevents the massive parallelization GPUs offer, forcing models to wait on a step-by-step chain. This bottleneck makes training on web-scale datasets painstakingly slow.



Numbers indicate min # of steps before a state can be computed

Introduction to Attention

The Beginning of the NLP revolution

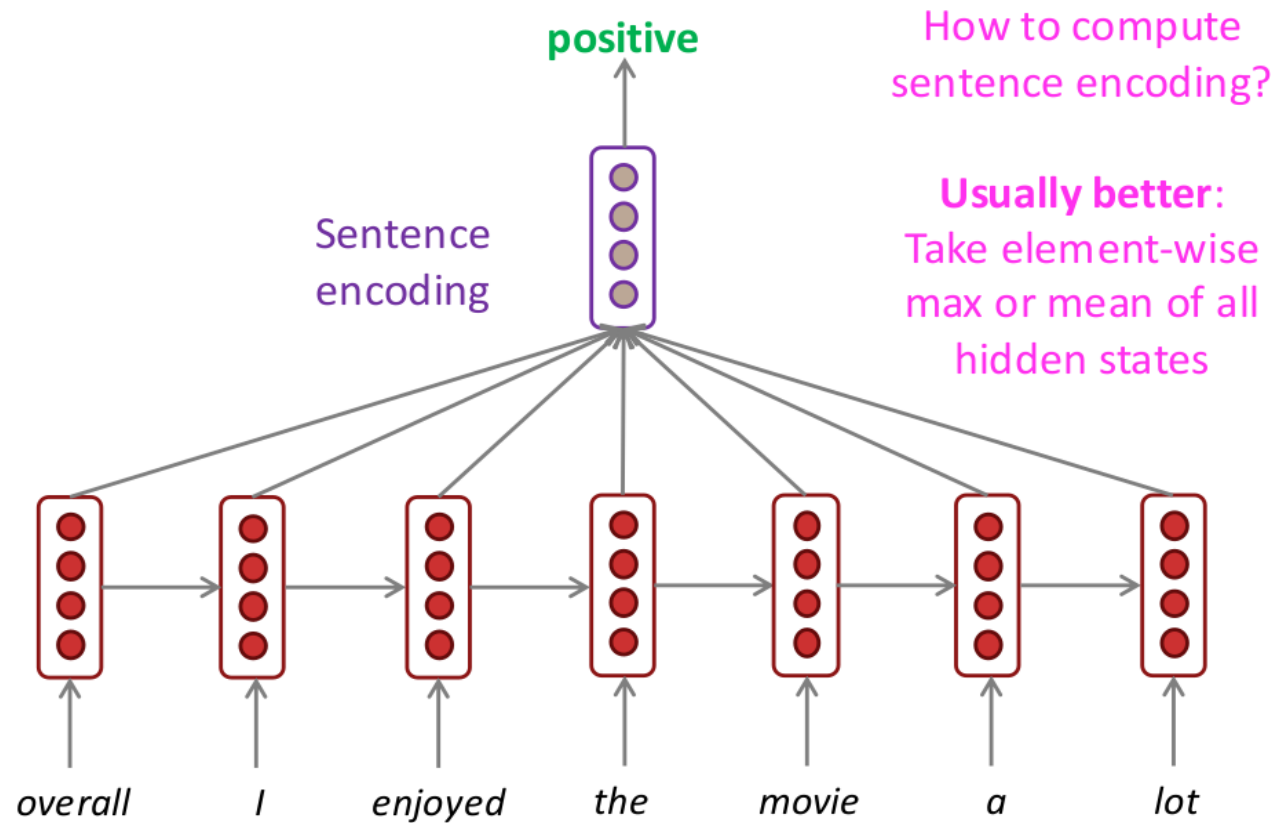
Attention

- **Attention** provides a solution to the bottleneck problem.
- **Core idea:** on each step of the decoder, *use direct connection to the encoder to focus on a particular part* of the source sequence



- First, we will show via diagram (no equations), then we will show with equations

Beyond the Last State: Pooling

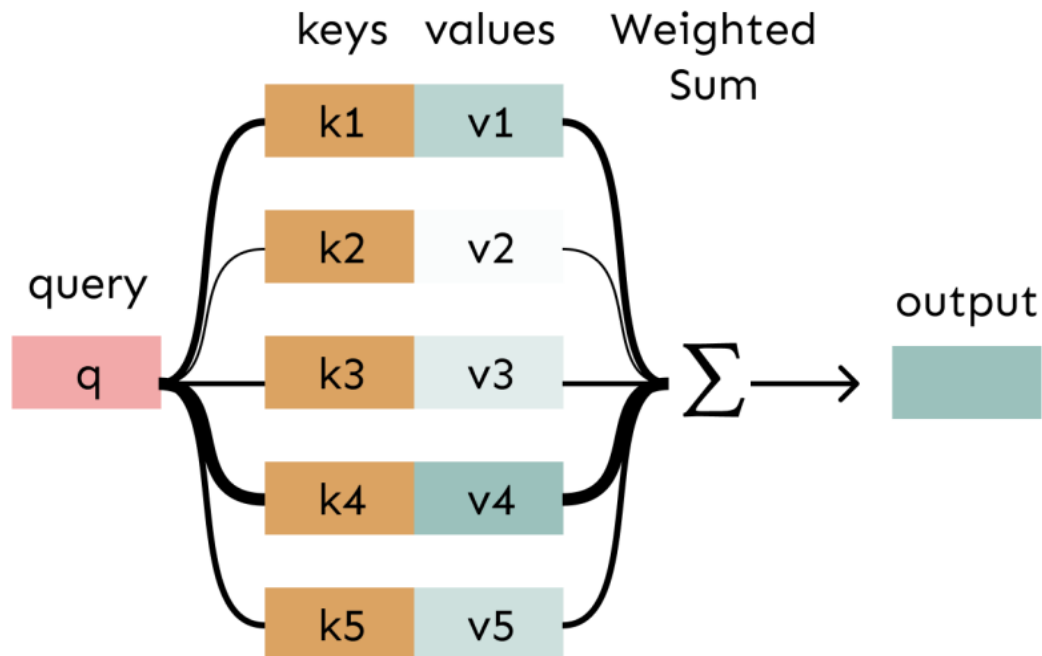


A basic fix for the bottleneck: average all encoder hidden states together. While better than just taking the last state, this "mean-pool" is a blunt instrument that mashes important words and filler into a generic blur.

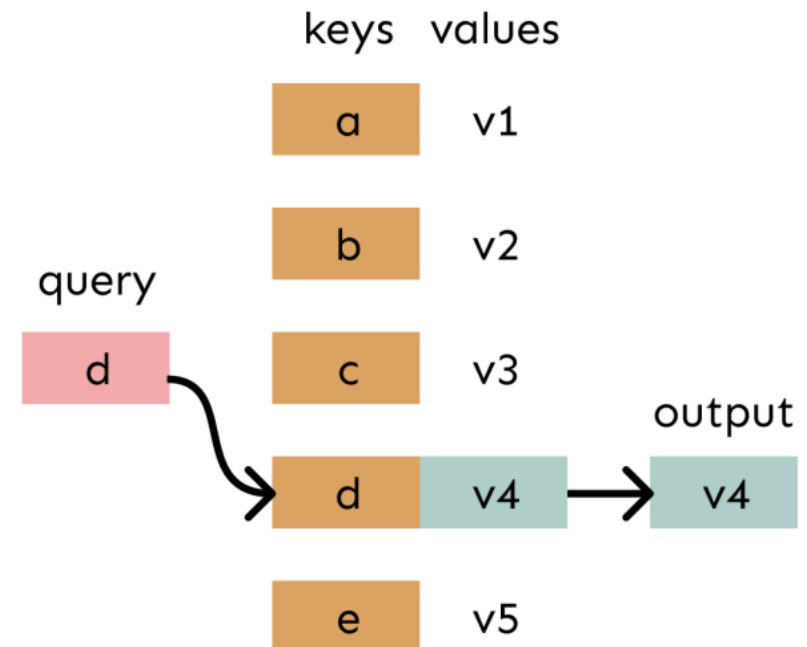
Attention: A soft Database Lookup

Attention is just a **weighted** average – this is very powerful if the weights are learned!

In **attention**, the **query** matches all **keys** *softly*, to a weight between 0 and 1. The keys' **values** are multiplied by the weights and summed.

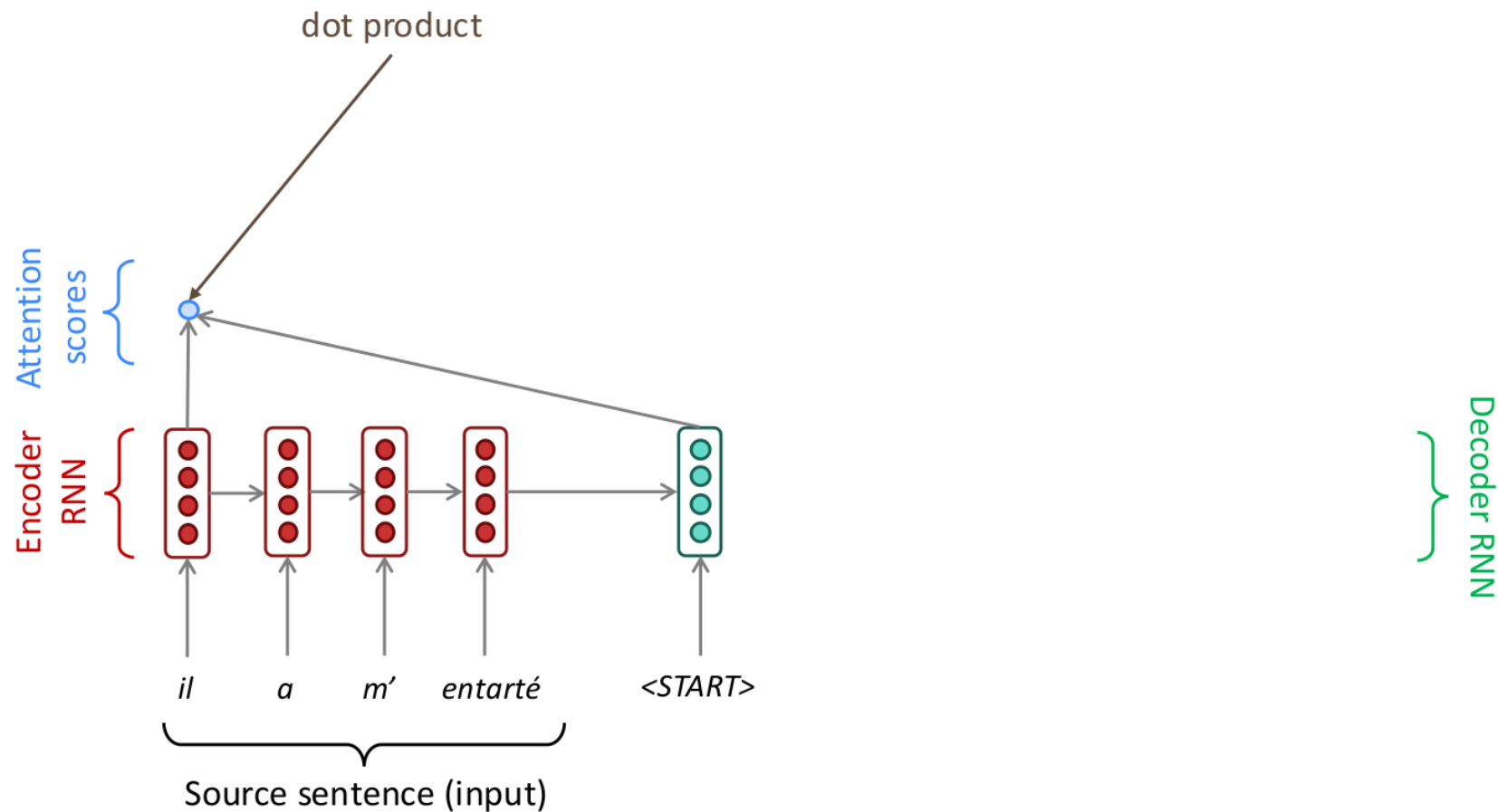


In a **lookup table**, we have a table of **keys** that map to **values**. The **query** matches one of the keys, returning its value.

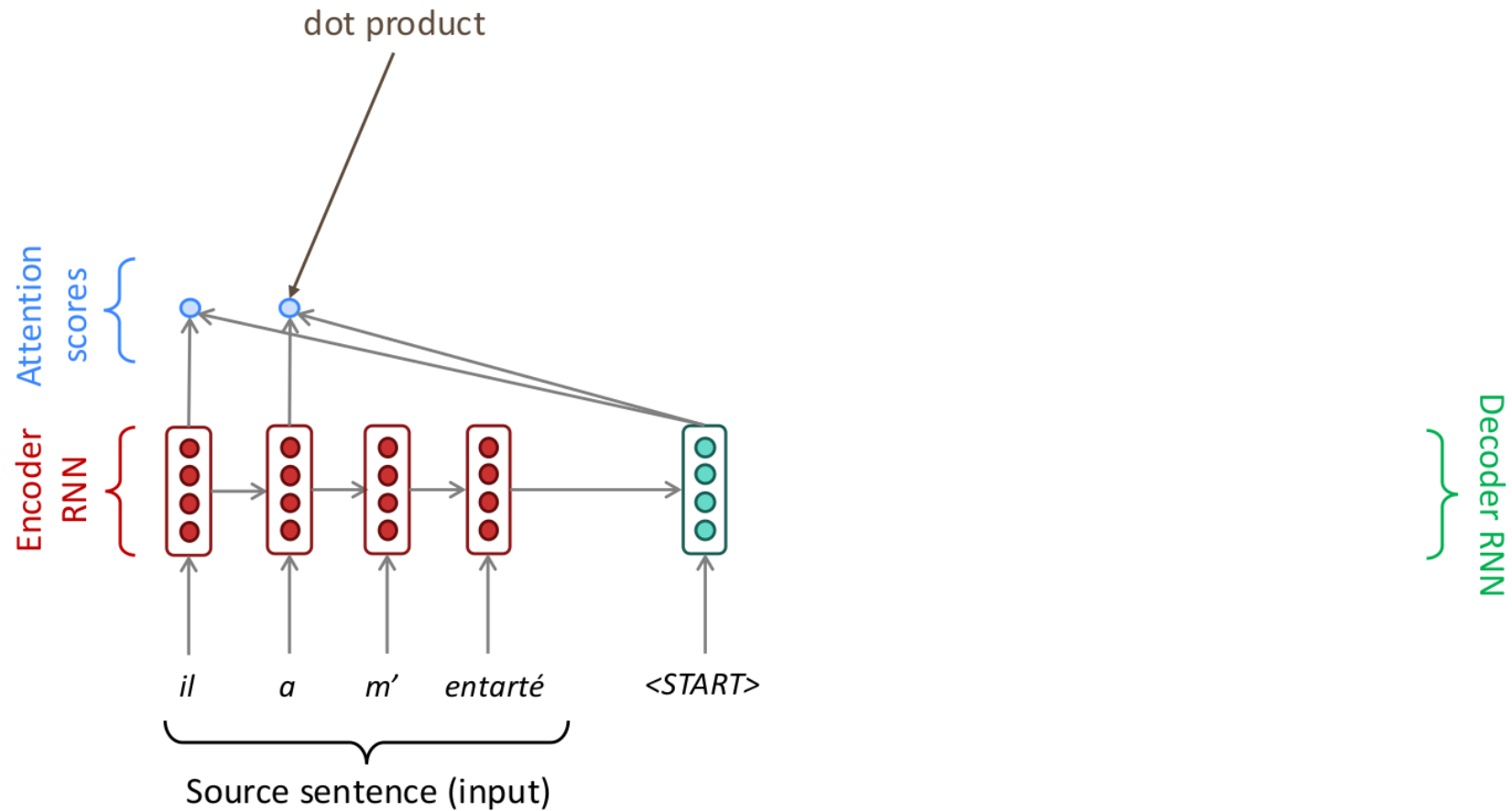


Attention scores: Step 1

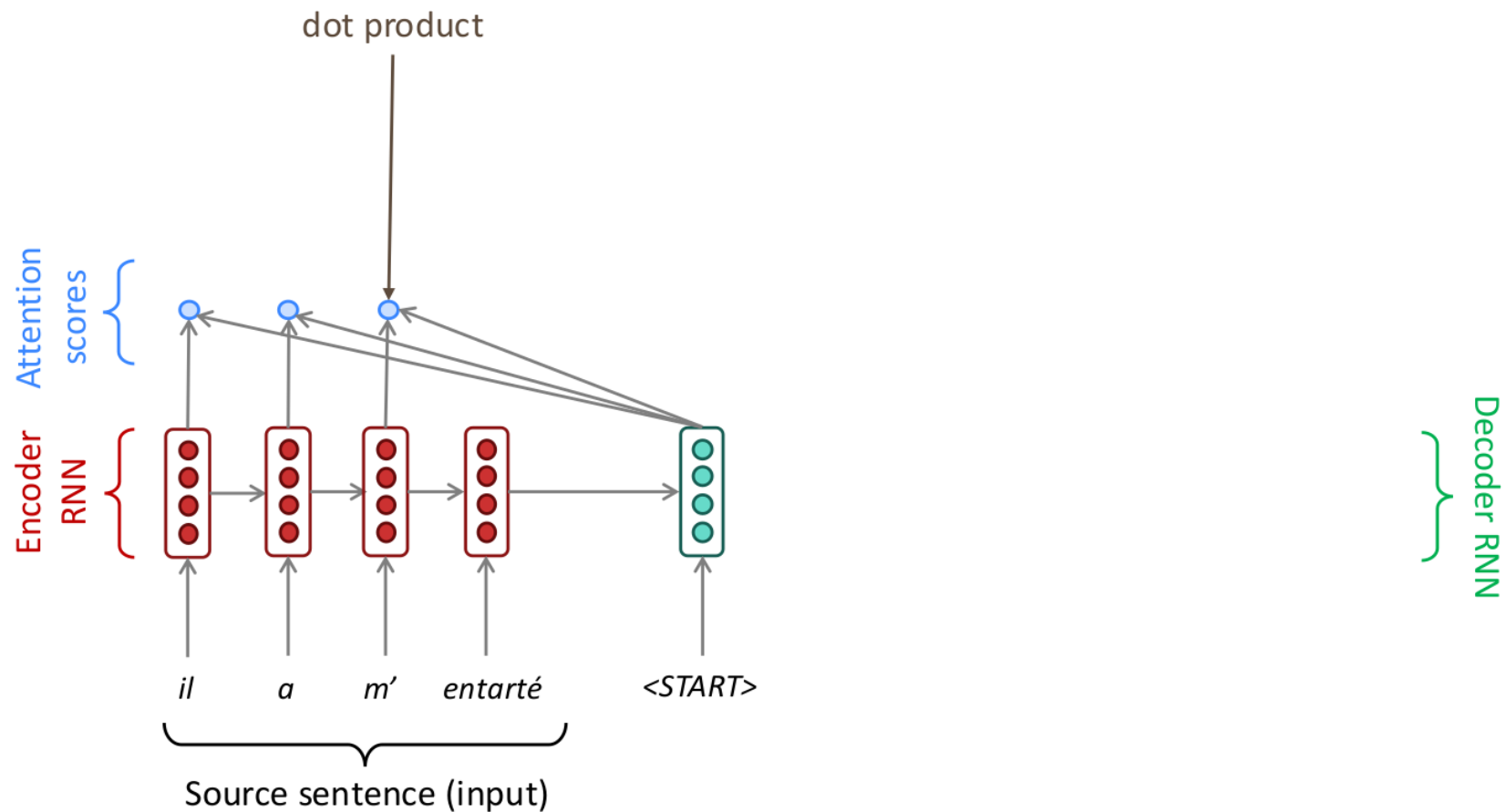
Core idea: on each step of the decoder, *use direct connection to the encoder* to *focus on a particular part* of the source sequence



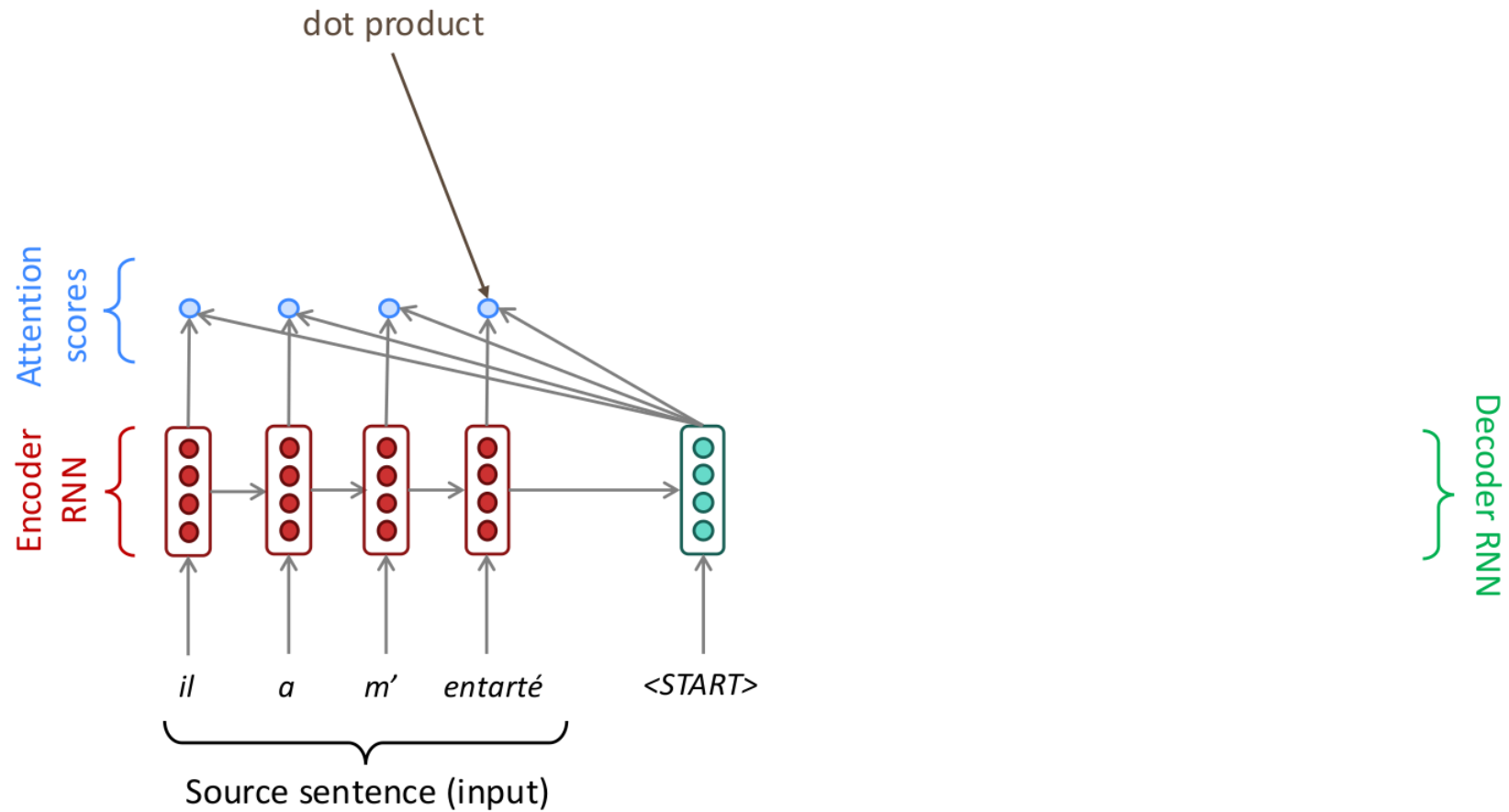
Attention scores: Step 2



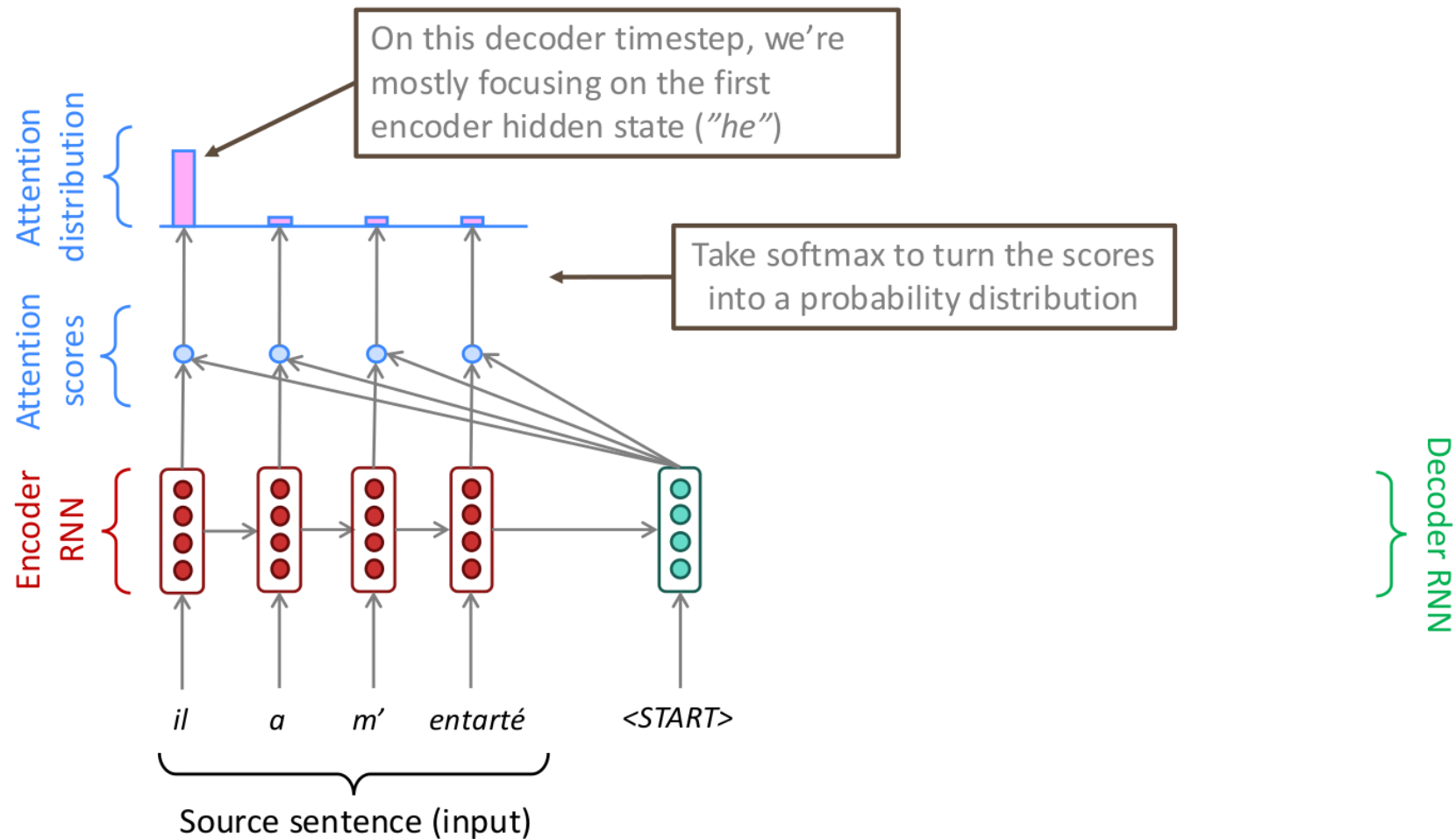
Attention scores: Step 3



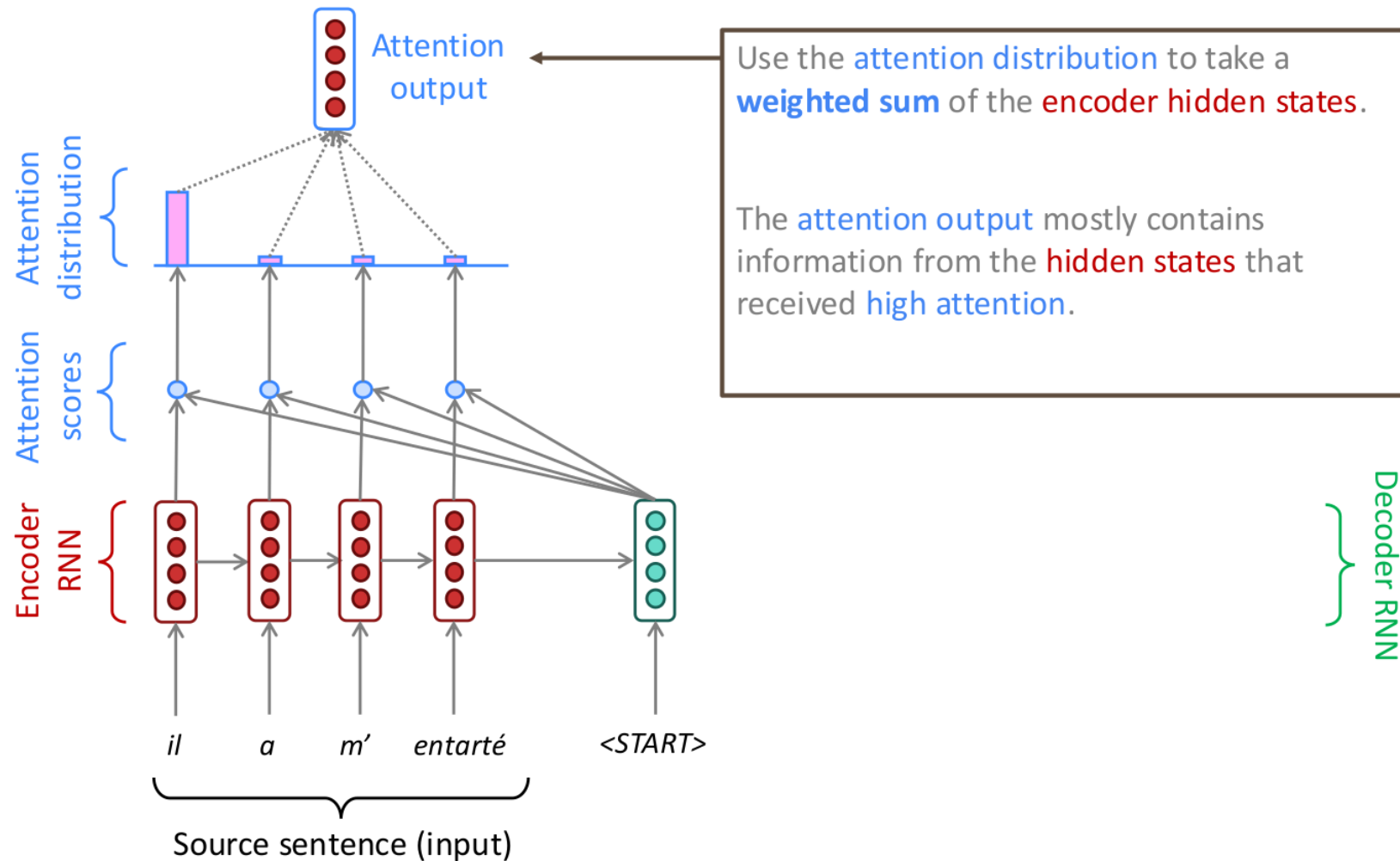
Attention scores: Step 4



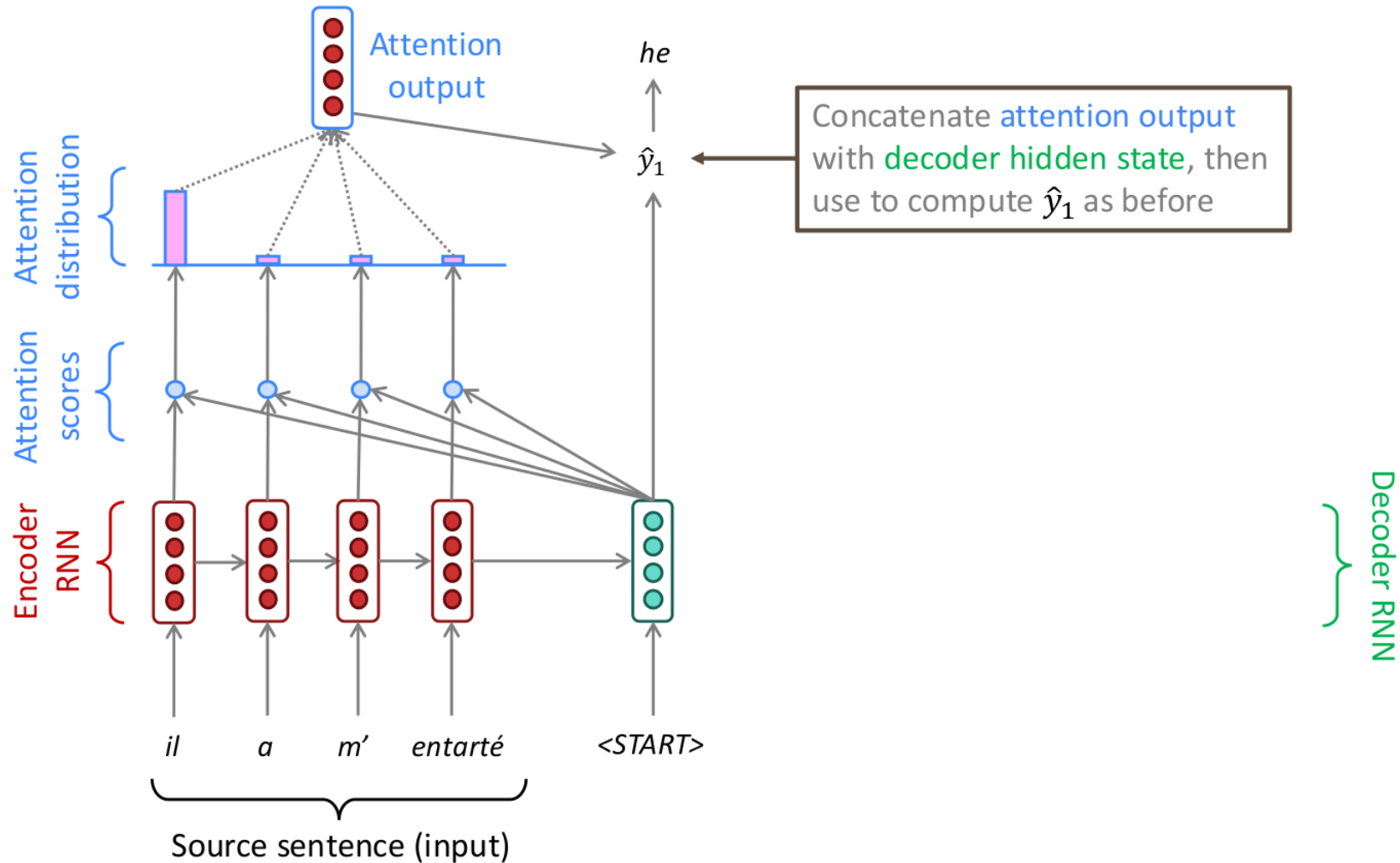
The Attention Distribution



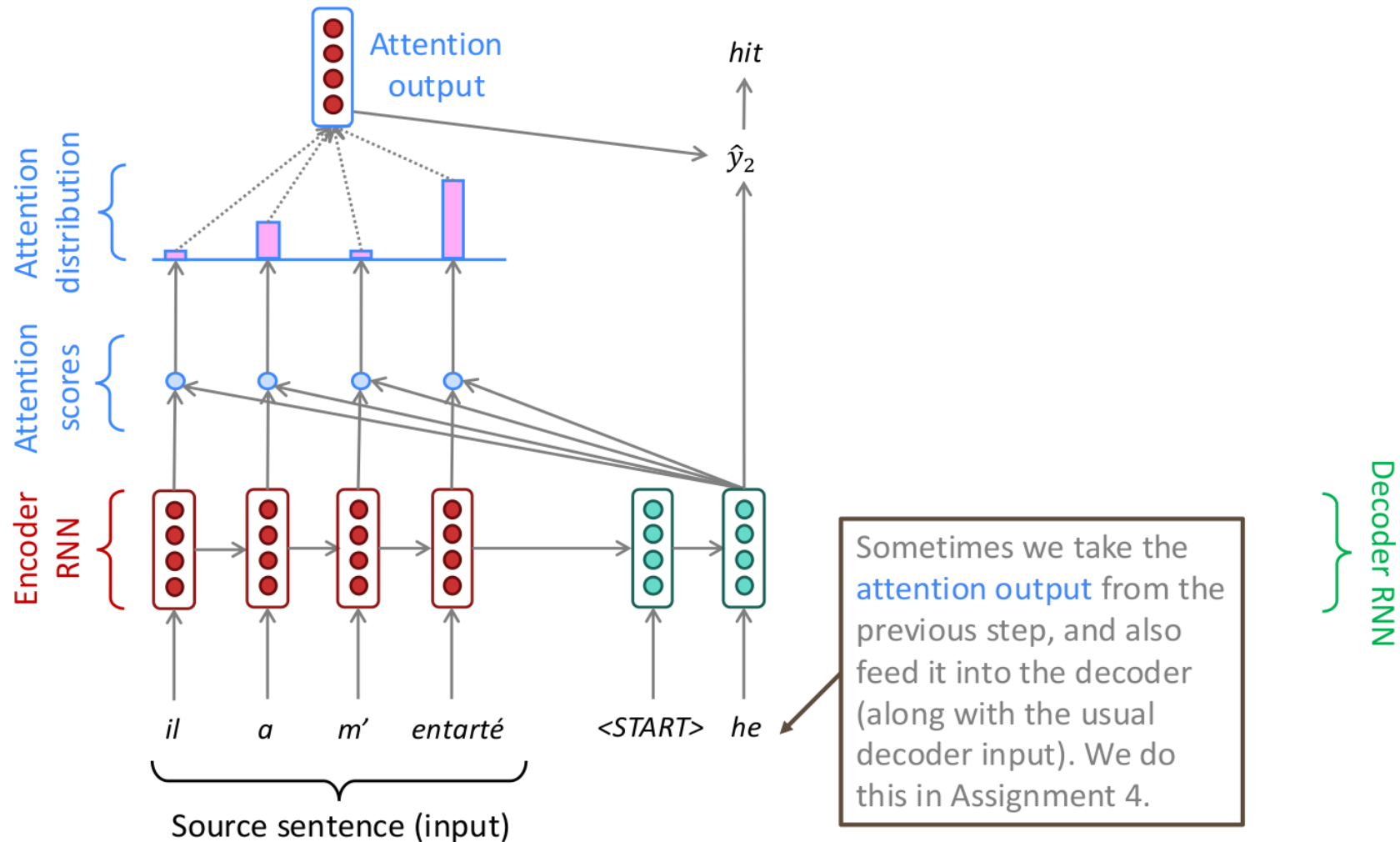
The Weighted Sum Output



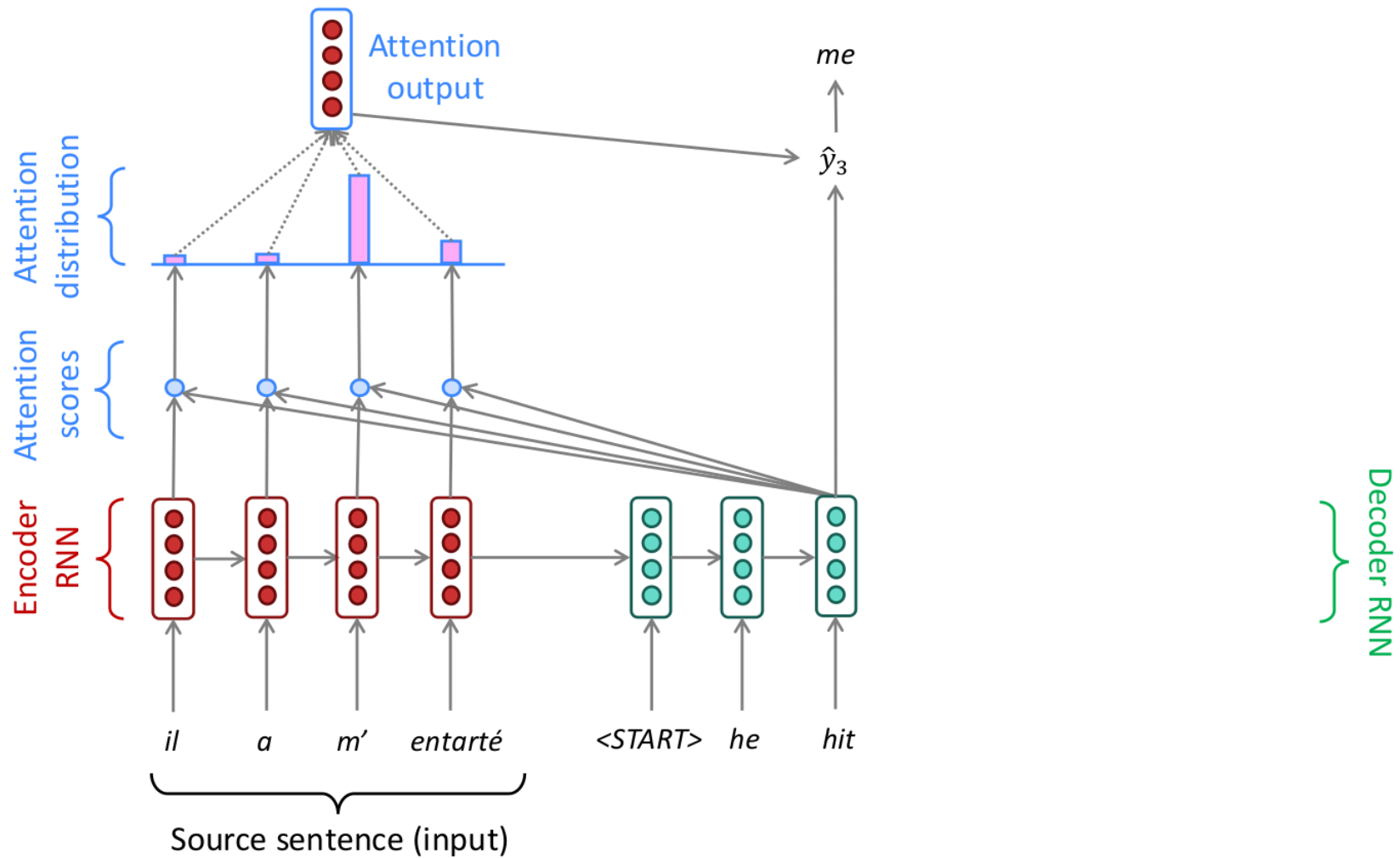
Concatenation and Prediction: Step 1



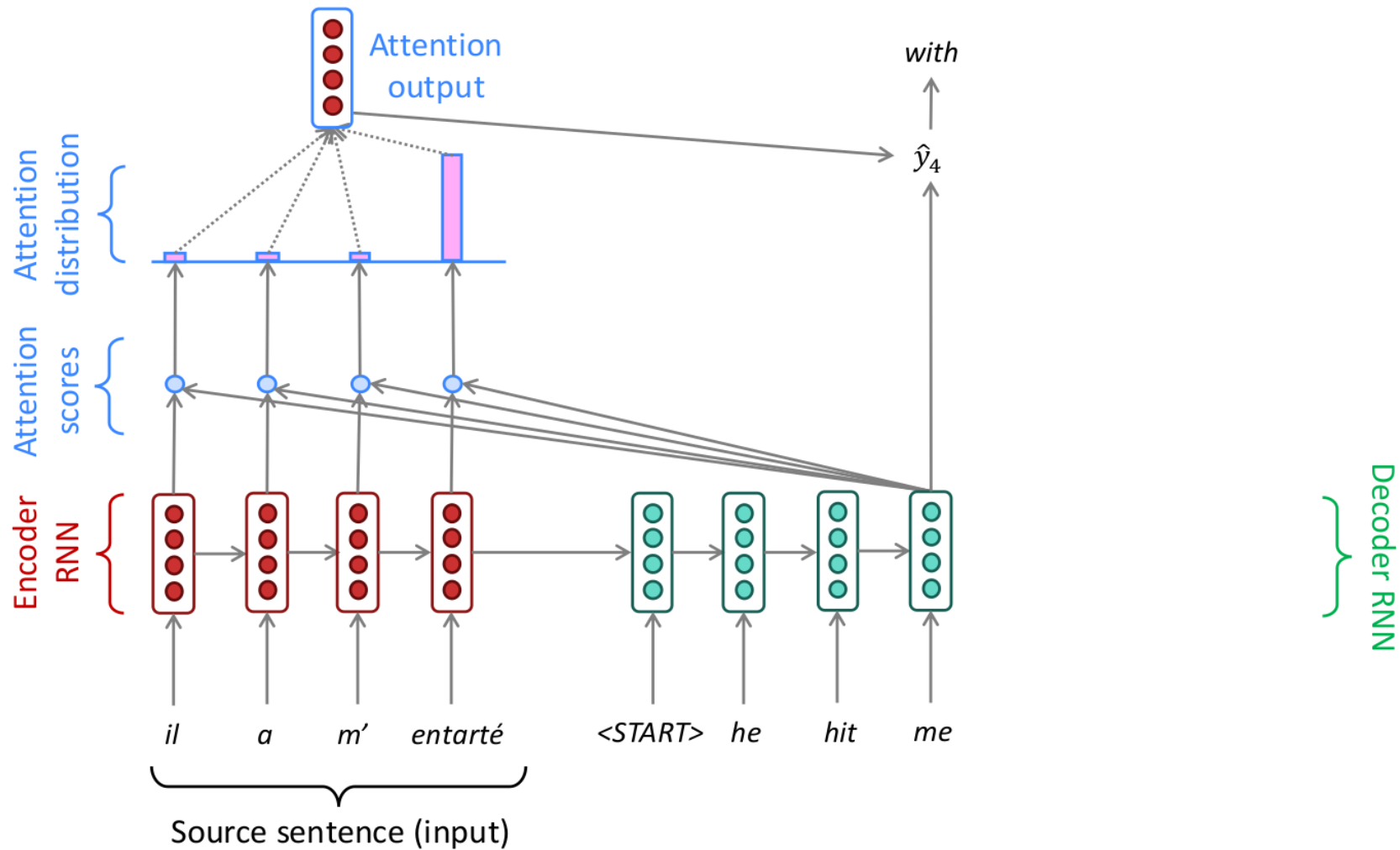
Concatenation and Prediction: Step 2



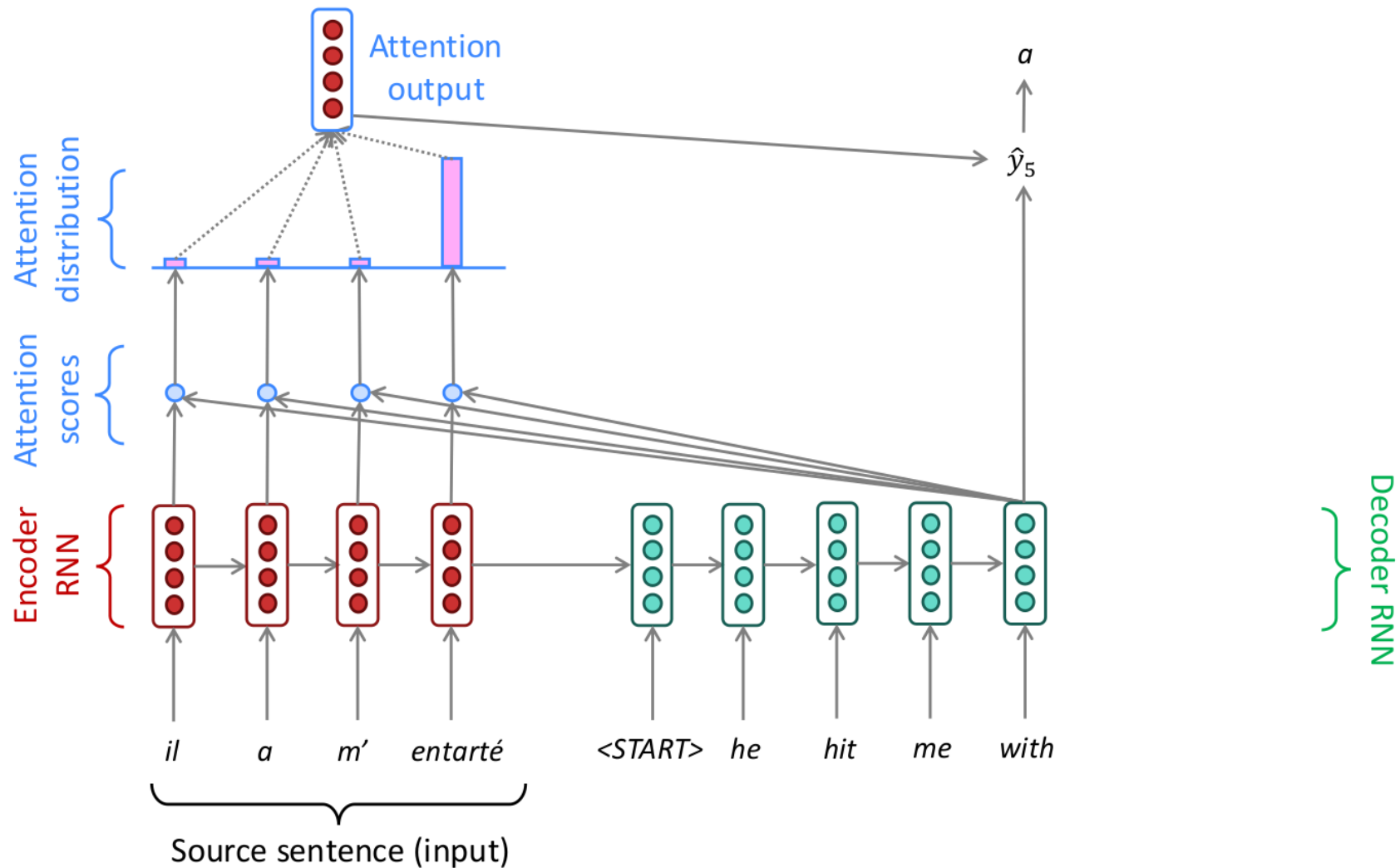
Concatenation and Prediction: Step 3



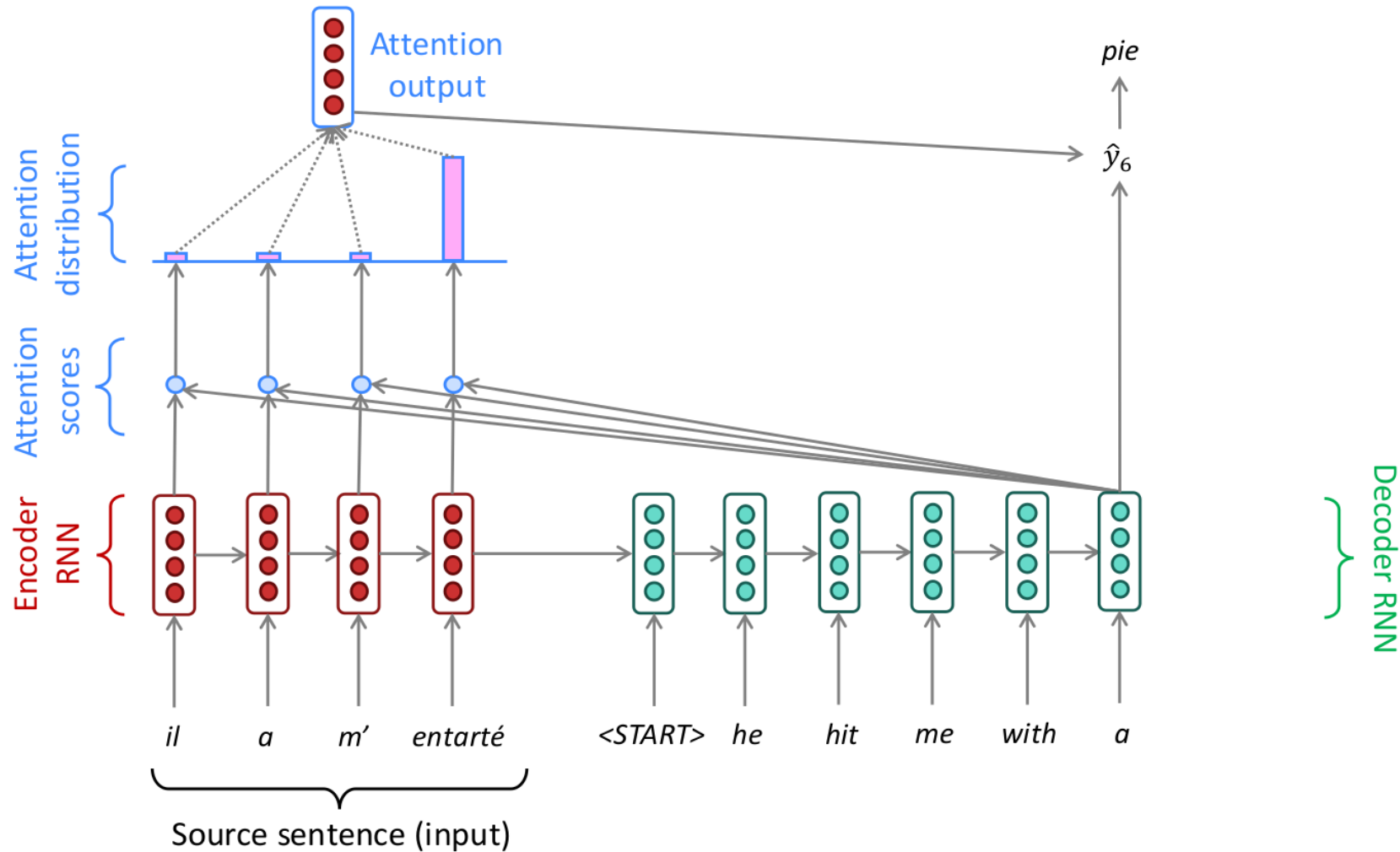
Concatenation and Prediction: Step 4



Concatenation and Prediction: Step 5



Concatenation and Prediction: Step 6



Attention in Equations

- We have encoder hidden states $h_1, \dots, h_N \in \mathbb{R}^h$
- On timestep t , we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$e^t = [s_t^T h_1, \dots, s_t^T h_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(e^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

$$a_t = \sum_{i=1}^N \alpha_i^t h_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

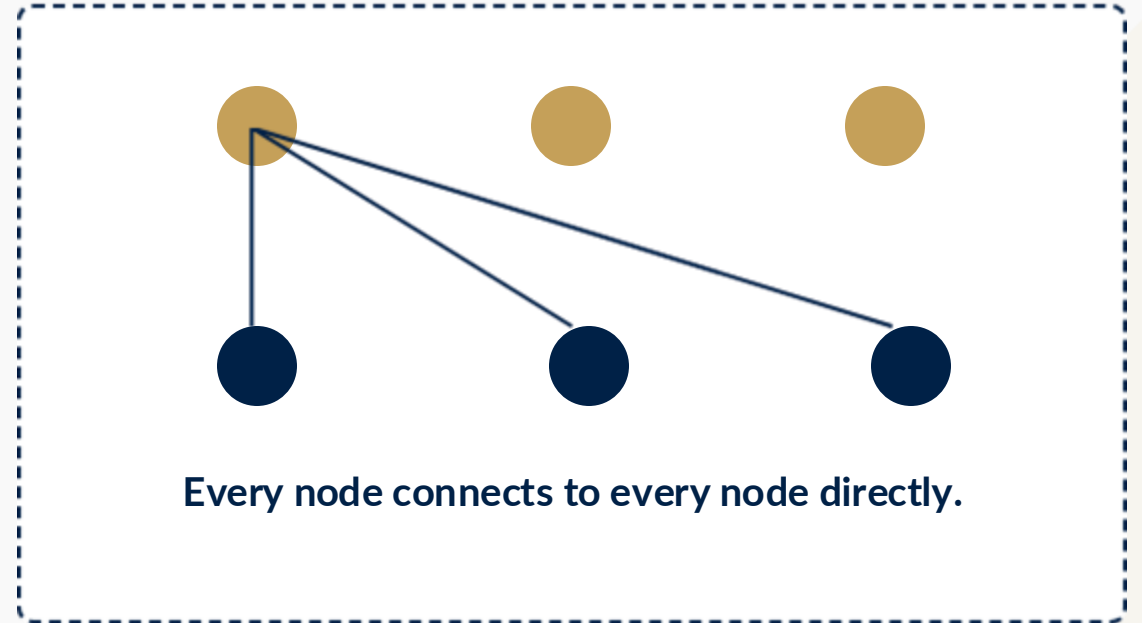
$$[a_t; s_t] \in \mathbb{R}^{2h}$$

Maximum Interaction Distance: $O(1)$

Parallelizable Shortcuts

Because Attention calculates similarities between the current state and **all** source words simultaneously, there are no "distant" words anymore.

The last word in a 100-word sentence is exactly **1 calculation step away** from the first word. This $O(1)$ shortcut completely obliterates the vanishing gradient problem associated with long sequences in traditional RNNs.

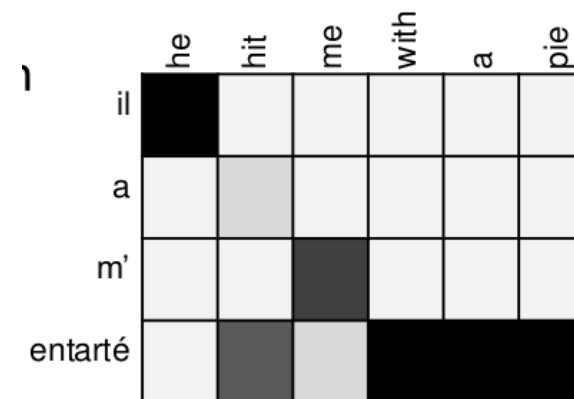


Why Attention is Great

- 🔑 **Improves Performance:** Drastically better translations.
- 👤 **Human-like Model:** Mimics how human translators dart their eyes back and forth across text.
- 🔒 **Solves the Bottleneck:** No need to compress an entire paragraph into a single array.
- 📈 **Fights Vanishing Gradients:** $O(1)$ direct connections.
- 🔍 **Interpretability:** Opens the "black box" of neural networks.

Alignment Matrix Heatmap

We can literally print out the softmax probabilities as a grid. If you want to know *why* the model translated a phrase a certain way, you can look at the grid and physically see the network staring at the exact source word.



A General DL technique

Attention is **not** just for sequence-to-sequence models or Machine Translation.

The Universal Definition:

*"Given a set of vector **values**, and a vector **query**, attention is a technique to compute a weighted sum of the values, dependent on the query."*

While invented as a "hack" to fix poor translation in RNNs, researchers quickly realized that comparing Queries to Keys to dynamically blend Values is a universally brilliant way to process arbitrary data formats.

The Future of Memory Manipulation

Selective Summary

The intuition behind Attention is that it acts as a "selective summary of the information contained in the values."

Attention fundamentally changed how Artificial Intelligence accesses memory. Instead of routing data through fixed, rigid paths (like convolutional filters) or sequential bottlenecks (like standard RNNs), Attention allows a neural network to **dynamically route its own data** based entirely on current context.

The Post-2010 Breakthrough

This mechanism is hailed as one of the most massive breakthroughs in AI history. This exact realization directly led to the invention of the **Transformer architecture**, which is the core engine making massive models like ChatGPT possible today.